

Java 24 — Generational Shenandoah

Java Version: 24
Year: 2025
Feature: Generational Shenandoah
UniqueClassName: GenerationalShenandoah

Definition

Generational Shenandoah is the generational mode of the **Shenandoah garbage collector**, introduced as a **production feature in Java 24**.

It divides the Java heap into two logical generations:

- **Young Generation** — contains newly created objects that are likely to become unreachable quickly.
- **Old Generation** — contains objects that survive multiple garbage-collection cycles.

The main idea is based on the **generational hypothesis**:

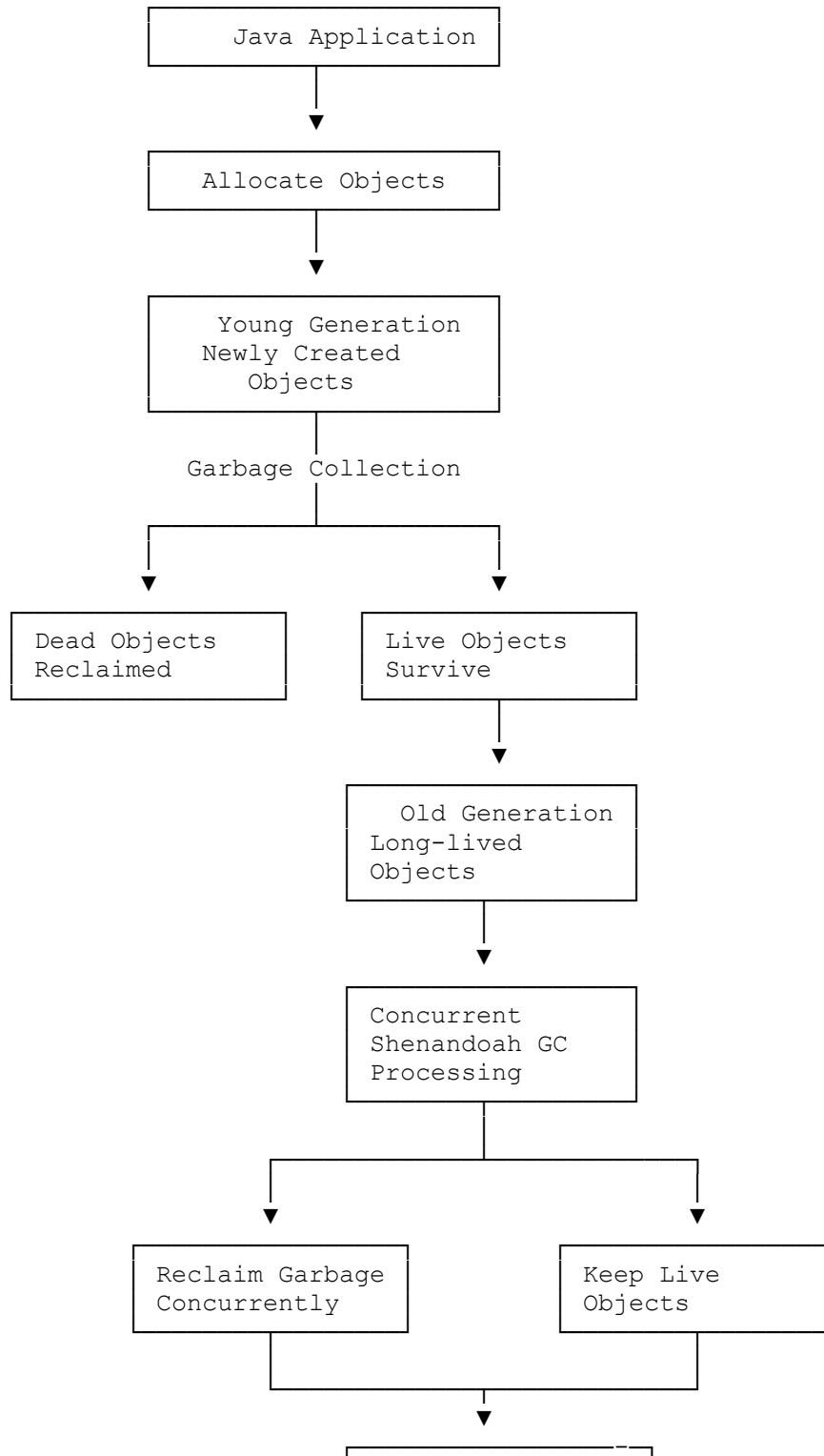
Most newly created objects become unreachable quickly, while objects that survive longer are more likely to remain alive.

Generational Shenandoah uses this observation to improve garbage-collection efficiency while preserving Shenandoah's primary goal of **very low pause times**.

Key Characteristics

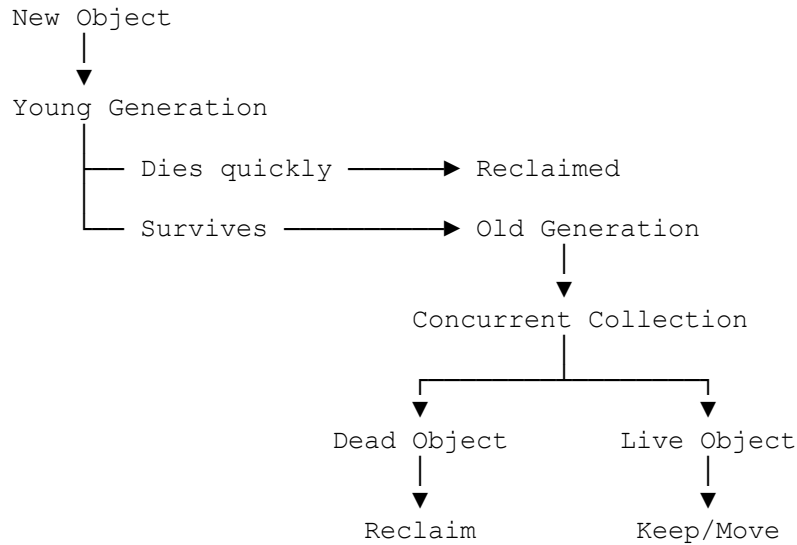
Characteristic	Description
Collector	Shenandoah GC
Generation model	Young + Old
Java 24 status	Production
Main goal	Improve throughput and reduce GC overhead
Pause-time goal	Very low
Heap compaction	Concurrent
Object movement	Concurrent evacuation
Suitable for	Large heaps and latency-sensitive applications
Activation	-XX:+UseShenandoahGC - XX:ShenandoahGCMode=generational

Flowchart



Application
Continues Running

Simplified Lifecycle



Java Code Example

Generational Shenandoah does **not** require special Java language syntax. Your application code is ordinary Java; the garbage collector is selected using JVM options.

```
public class GenerationalShenandoah {

    public static void main(String[] args) {

        System.out.println("Generational Shenandoah Demo");

        for (int i = 0; i < 1_000_000; i++) {

            // Short-lived objects
            byte[] temporaryObject = new byte[1024];

            // Use the object
            temporaryObject[0] = (byte) i;

            if (i % 100_000 == 0) {
                System.out.println("Created objects: " + i);
            }

        }

        System.out.println("Application completed.");

    }

}
```

Running with Generational Shenandoah

Compile:

```
javac GenerationalShenandoah.java
```

Run:

```
java -XX:+UseShenandoahGC \  
    -XX:ShenandoahGCMode=generational \  
    GenerationalShenandoah
```

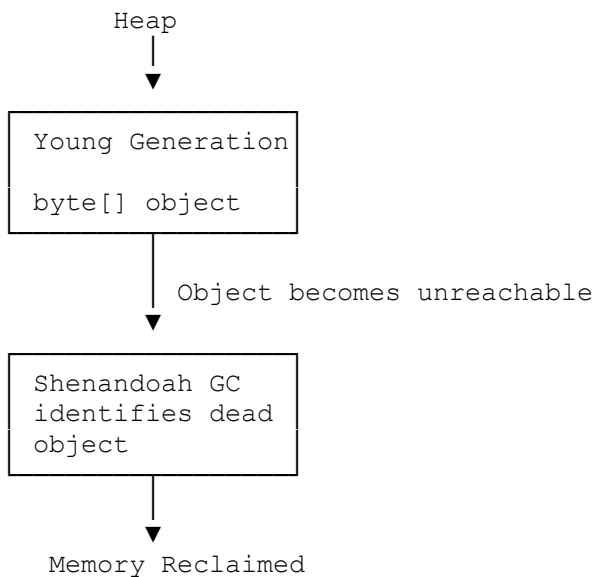
You can also enable GC logging:

```
java -XX:+UseShenandoahGC \  
    -XX:ShenandoahGCMode=generational \  
    -Xlog:gc \  
    GenerationalShenandoah
```

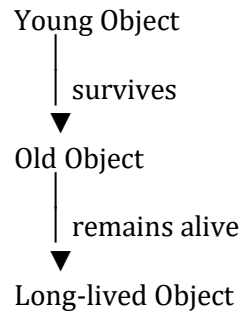
What happens internally?

```
byte[] temporaryObject = new byte[1024];
```

creates an object on the heap.

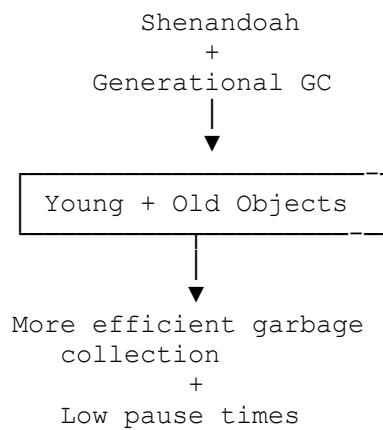


If an object remains alive through collections, it can eventually be treated as an **old-generation object**:



Important Point

Generational Shenandoah combines two ideas:



So, **Generational Shenandoah is not a new Java language feature**. It is a **JVM garbage-collection enhancement** that organizes Shenandoah's heap management around object generations to improve performance and GC efficiency.

Java 24 — Compact Object Headers

Java Version: 24

Year: 2025

Feature: Compact Object Headers

UniqueClassName: CompactObjectHeaders

Definition

Compact Object Headers is a **preview JVM feature introduced in Java 24** that reduces the memory footprint of object headers on 64-bit platforms.

Every Java object has an internal **object header** maintained by the JVM. The header contains JVM metadata such as information needed for object identity, locking, and class information.

Traditionally, an object header could consume **12 bytes or more**, depending on the JVM configuration. Compact Object Headers redesigns this representation so that, in many cases, the header can be reduced to approximately **8 bytes**.

Traditional Object Header

Mark Word Lock / identity / GC metadata
Class Pointer Object's class information

≈ 12 bytes

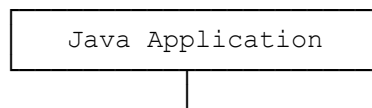
Compact Object Header

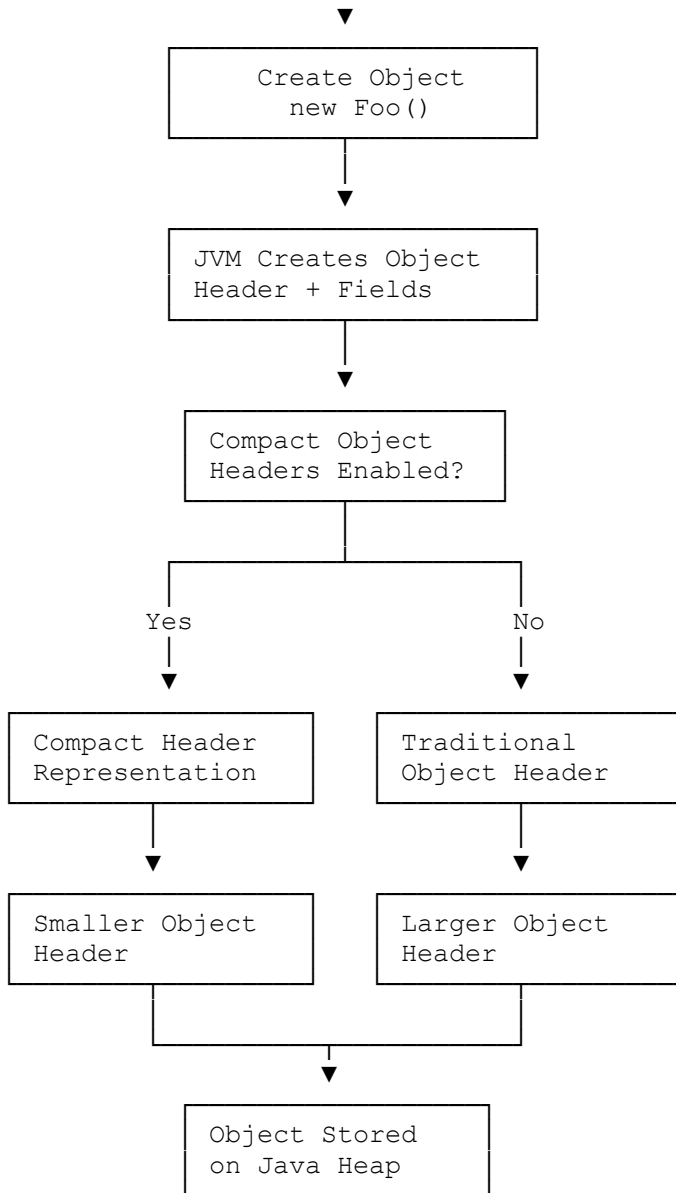
Compact Header Class + metadata information
--

≈ 8 bytes

The reduction is particularly valuable for applications that create **millions or billions of small objects**, because even a few bytes saved per object can significantly reduce heap consumption.

Flowchart





Why It Matters

Suppose an application creates **100 million small objects**.

If the object header is reduced by roughly 4 bytes:

100,000,000 objects × 4 bytes
↓
400,000,000 bytes
↓
≈ 381 MiB saved

The exact memory savings depend on the JVM, object layout, alignment, object type, and runtime configuration.

Java Code Example

Compact Object Headers **do not** introduce new Java syntax. The Java application can be completely ordinary:

```
public class CompactObjectHeaders {

    static class Customer {
        int id;
        String name;

        Customer(int id, String name) {
            this.id = id;
            this.name = name;
        }
    }

    public static void main(String[] args) {

        Customer customer =
            new Customer(101, "Anurag");

        System.out.println("Customer ID: " + customer.id);
        System.out.println("Customer Name: " + customer.name);
    }
}
```

Compile:

```
javac CompactObjectHeaders.java
```

The important part is **how the JVM is started**, rather than any special Java syntax.

Enabling Compact Object Headers

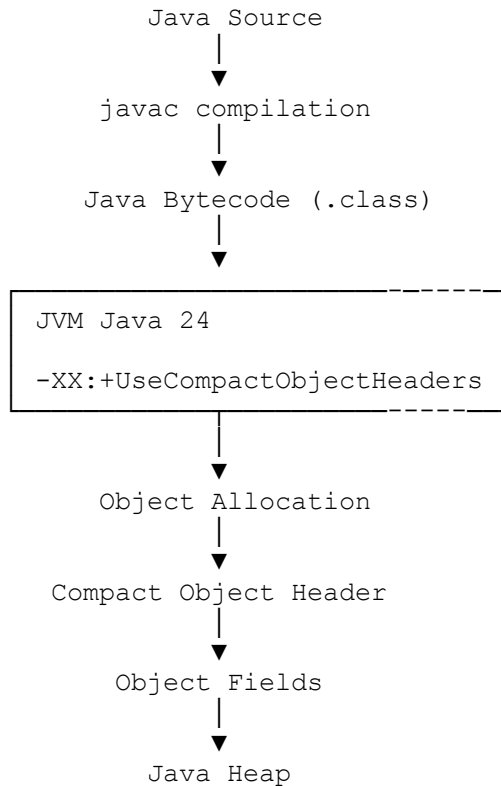
In Java 24, Compact Object Headers is a **preview feature** and can be enabled using the JVM option:

```
java -XX:+UseCompactObjectHeaders CompactObjectHeaders
```

For example:

```
java -XX:+UseCompactObjectHeaders \
  CompactObjectHeaders
```

The Java program itself remains unchanged:



Traditional vs Compact

Aspect	Traditional Headers	Compact Object Headers
Java syntax	No change	No change
JVM feature	Existing representation	Java 24 preview
Object header	Larger	Smaller
Typical target	General JVM usage	Memory-efficient object layouts
Main benefit	Compatibility/current design	Reduced object memory footprint
Especially useful for	General applications	Applications with many small objects
Configuration	Default	-XX:+UseCompactObjectHeaders

Key Idea

Traditional:

Object

- └─ Larger Header
- └─ Fields

Compact:

Object

- └─ Smaller Header
- └─ Fields

↓

Less memory per object

↓

More objects fit in the same heap

↓

Potentially better memory efficiency

In one sentence: Java 24 Compact Object Headers reduces the memory overhead associated with each Java object by using a more compact JVM object-header representation, which can be especially beneficial for applications containing very large numbers of small objects.

Java 24 — Prepare to Restrict the Use of JNI

Java Version: 24

Year: 2025

Feature: Prepare to Restrict the Use of JNI

UniqueClassName: RestrictJNI

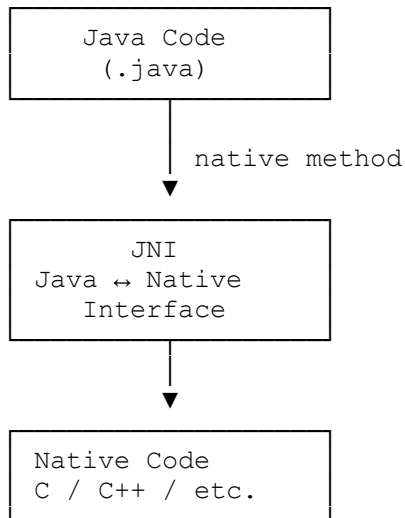
Definition

Prepare to Restrict the Use of JNI is a **Java 24 warning/deprecation-related change** designed to prepare developers for stronger restrictions on the use of the **Java Native Interface (JNI)** in future Java releases.

JNI (Java Native Interface) allows Java code to interact with native code written in languages such as **C** and **C++**.

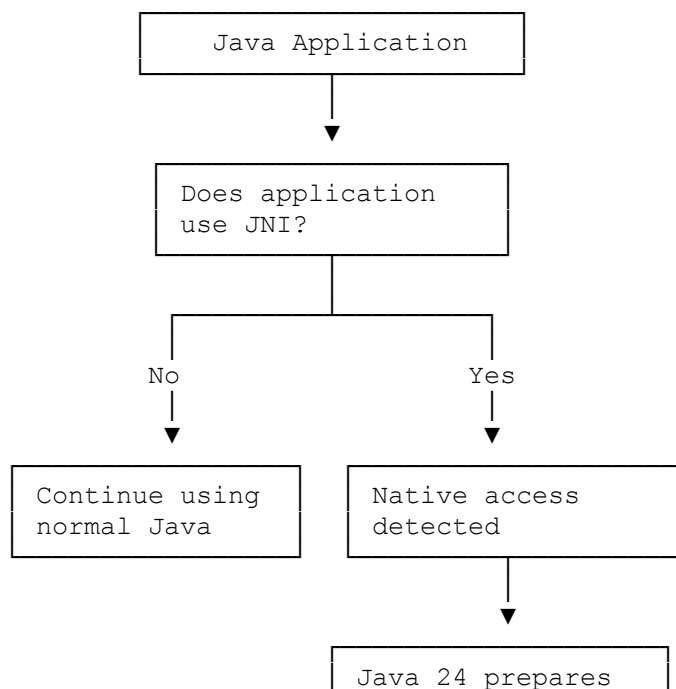
The Java Platform increasingly favors safer mechanisms such as the **Foreign Function & Memory (FFM) API** for interoperability with native code. Java 24 therefore begins preparing applications for a future in which native access through JNI may require explicit permission.

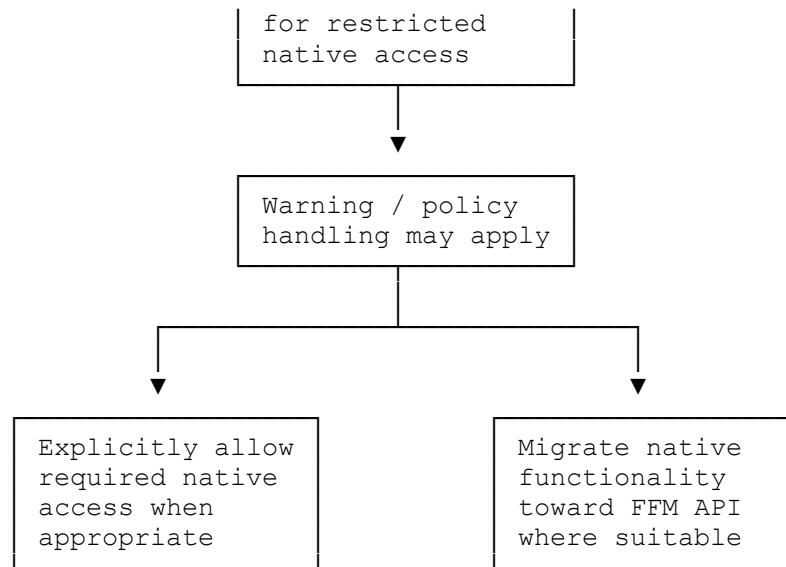
Traditional JNI Model



The change is intended to make applications **explicit about their use of native code**, improving security and preparing the platform for stronger encapsulation.

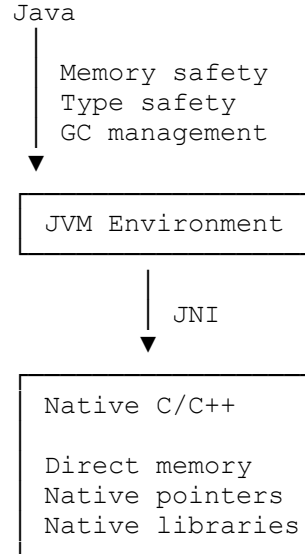
Flowchart





Why Is JNI Being Restricted?

JNI is powerful, but native code operates outside many of Java's safety guarantees.



Native code can interact directly with memory and the operating system. Consequently, unrestricted native access can weaken the JVM's isolation and security model.

Java's newer **FFM API** provides a more structured mechanism for calling native functions and accessing native memory.

Java Code Example — JNI

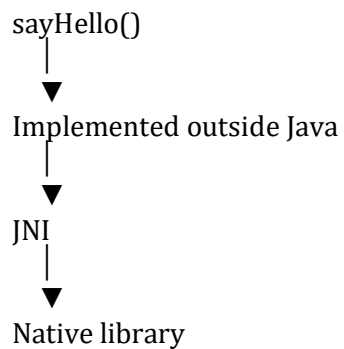
A simple JNI example consists of a Java class declaring a native method:

```
public class RestrictJNI {  
  
    // Declares a method implemented in native code  
    public static native void sayHello();  
  
    static {  
        System.loadLibrary("nativehello");  
    }  
  
    public static void main(String[] args) {  
  
        System.out.println("Calling native code...");  
  
        sayHello();  
  
        System.out.println("Returned to Java.");  
    }  
}
```

The important line is:

```
public static native void sayHello();
```

It tells the JVM:

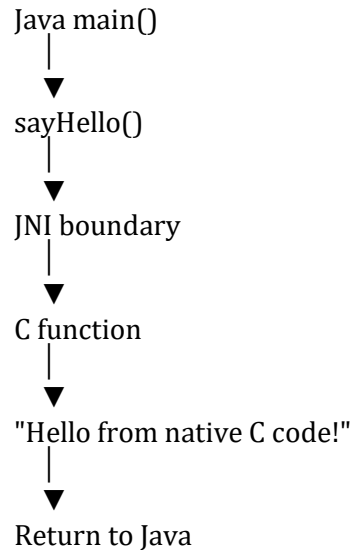


A corresponding native implementation could be written in C:

```
#include <jni.h>  
#include <stdio.h>  
  
JNIEXPORT void JNICALL
```

```
Java_RestrictJNI_sayHello(JNIEnv *env, jclass clazz) {  
    printf("Hello from native C code!\n");  
}
```

The Java application therefore crosses the Java/native boundary:

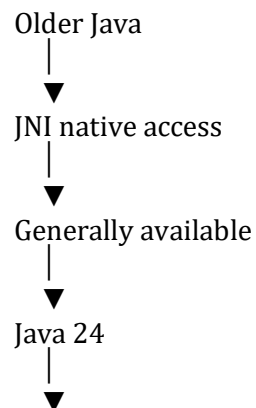


Java 24 Perspective

The important point is that **Java 24 does not simply remove JNI**.

Instead, Java 24 is part of a longer transition toward requiring **explicit authorization for native access**.

Conceptually:



Prepare for stronger restrictions



Future Java releases



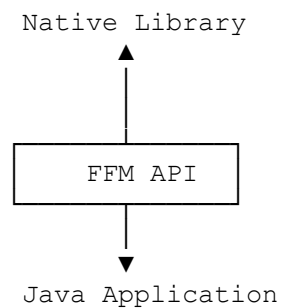
Explicit native-access authorization

For applications that intentionally use JNI, developers should therefore be aware of the JVM's native-access restrictions and warnings rather than assuming native access will remain unrestricted indefinitely.

Alternative: Foreign Function & Memory API

For many new native-integration projects, Java's **Foreign Function & Memory API (FFM)** is the preferred modern alternative to JNI.

Conceptually:



Instead of:

Java



JNI



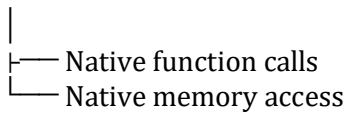
C/C++

modern Java can use:

Java



Foreign Function & Memory API



Summary

Aspect	Prepare to Restrict JNI
Java version	24
Year	2025
Feature type	JVM/platform migration preparation
Main technology	JNI
Purpose	Prepare applications for stronger native-access restrictions
Why	Improve security and platform integrity
New Java syntax	No
JNI removed?	No
Modern alternative	Foreign Function & Memory API
Main developer action	Review JNI/native access and prepare for explicit authorization

In one sentence: Java 24's "Prepare to Restrict the Use of JNI" is a migration step that makes unrestricted JNI/native access less future-proof and encourages applications to explicitly manage native access and, where appropriate, migrate to the Foreign Function & Memory API.

Java 24 — Late Barrier Expansion for G1

Java Version: 24

Year: 2025

Feature: Late Barrier Expansion for G1

UniqueClassName: LateBarrierExpansionG1

Definition

Late Barrier Expansion for G1 is a JVM implementation improvement introduced in Java 24 for the **Garbage-First (G1) Garbage Collector**.

It changes **when certain GC barriers are expanded into their lower-level machine-code operations** during JIT compilation.

A **GC barrier** is a small piece of code inserted around certain object/reference operations so that the garbage collector can maintain information needed for safe and efficient garbage collection.

Previously, some barriers were expanded relatively early during the compiler pipeline. With **late barrier expansion**, G1 can keep barriers in a more abstract form for longer and expand them later, allowing the compiler more opportunity to optimize the surrounding code.

Main idea

Traditional approach:

```
Java Code
↓
JIT Compilation
↓
Early Barrier Expansion
↓
Optimization
↓
Machine Code
```

Java 24 improvement:

```
Java Code
↓
JIT Compilation
↓
Keep G1 Barrier Abstract
↓
More Compiler Optimization
↓
Late Barrier Expansion
↓
```

Machine Code

The feature is primarily an **internal HotSpot compiler/G1 implementation change**. It does **not** introduce a new Java language construct.

What Is a G1 Barrier?

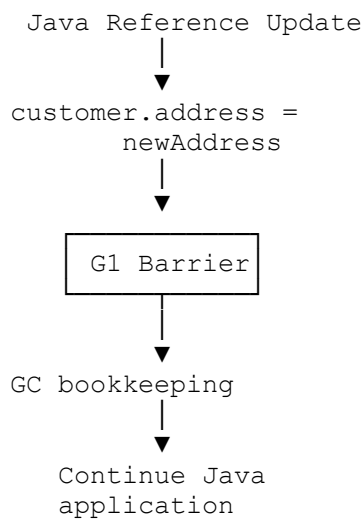
G1 uses barriers to help track references and heap changes while performing concurrent and incremental garbage collection.

For example, when Java code modifies an object reference:

```
customer.address = newAddress;
```

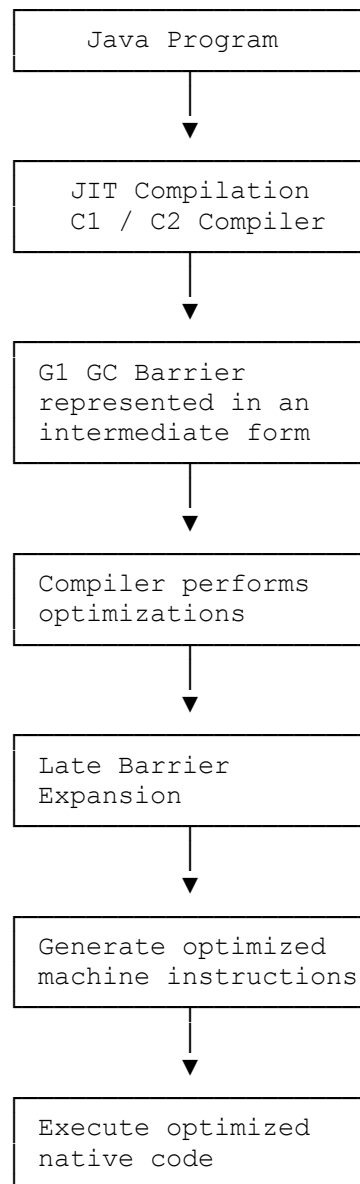
the JVM may need GC-related bookkeeping associated with that reference update.

Conceptually:



The actual barriers and generated machine code are JVM implementation details; Java source code does not directly invoke them.

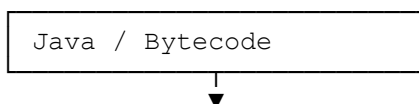
Flowchart

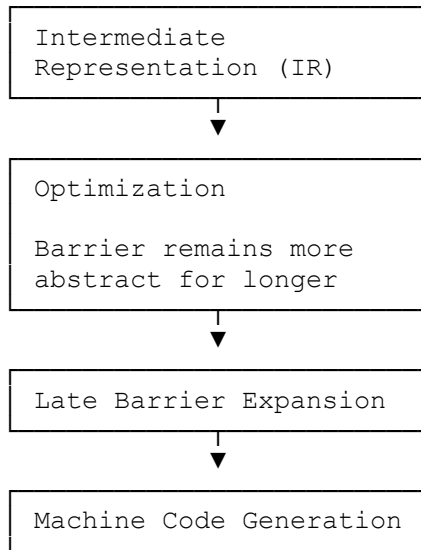


Why "Late"?

The word **late** refers to the point in the JIT compiler pipeline at which the abstract G1 barrier is converted into its detailed low-level implementation.

Compiler Pipeline





This separation can make it easier for the compiler to optimize code around GC barriers and can reduce unnecessary work during compilation.

Java Code Example

There is **no special Java API** for Late Barrier Expansion. The same Java application can run with G1.

```
public class LateBarrierExpansionG1 {  
    static class Customer {  
        String name;  
        Customer next;  
  
        Customer(String name) {  
            this.name = name;  
        }  
    }  
  
    public static void main(String[] args) {  
        Customer head = null;  
  
        for (int i = 0; i < 1_000_000; i++) {  
            Customer customer =  
                new Customer("Customer-" + i);  
  
            // Reference update.
```

```

        // G1 may perform GC barrier
        // bookkeeping internally.
        customer.next = head;

        head = customer;

        // Periodically discard the chain.
        if (i % 10_000 == 0) {
            head = null;
        }
    }

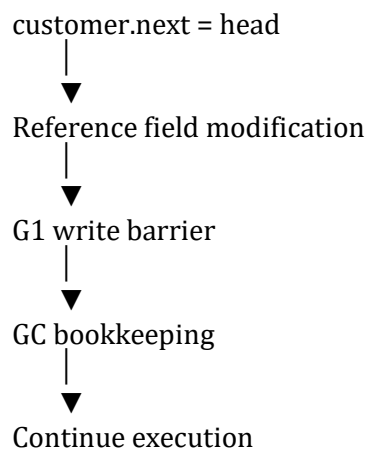
    System.out.println("Application completed.");
}
}

```

The important operation from the G1 perspective is the **reference update**:

`customer.next = head;`

Conceptually:



The programmer does **not** write the barrier explicitly.

Running with G1

G1 is the default collector on many modern Java configurations, but it can be explicitly selected:

```
java -XX:+UseG1GC LateBarrierExpansionG1
```

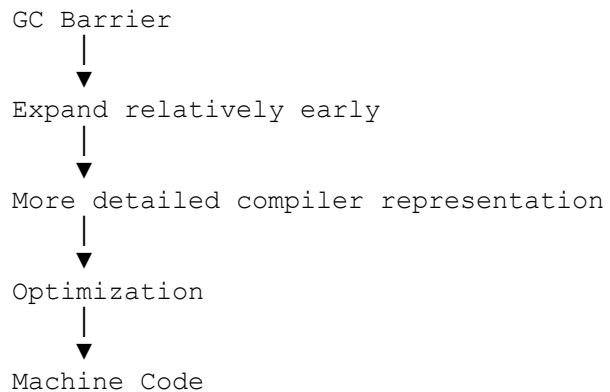
With GC logging:

```
java -XX:+UseG1GC -Xlog:gc LateBarrierExpansionG1
```

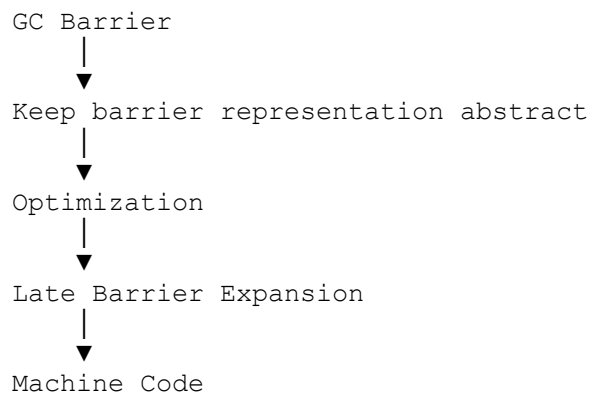
The **Late Barrier Expansion** mechanism is handled internally by the JVM; there is normally **no separate command-line switch that application developers need to enable** for the feature.

Before vs. Java 24 Improvement

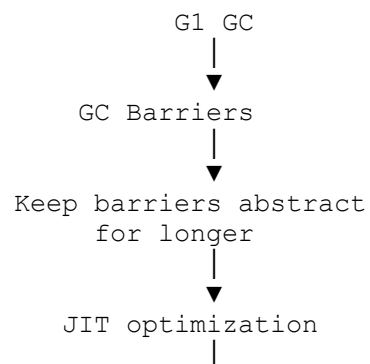
Earlier compiler approach

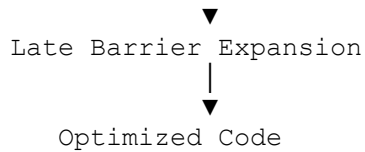


Java 24 approach



Overall Concept





Benefits

Benefit	Explanation
Better compiler optimization	Barriers can remain abstract while more optimization takes place.
Reduced compiler complexity	Barrier expansion can be separated from earlier compiler phases.
Potentially better generated code	The compiler has more information when barriers are finally expanded.
G1-specific	The change concerns G1's GC barriers and HotSpot compilation.
No source-code changes	Existing Java applications can benefit without changing their Java code.

Key Point

Late Barrier Expansion for G1 is not a Java programming feature like records, pattern matching, or virtual threads. It is an **under-the-hood HotSpot/G1 compiler optimization** introduced in Java 24 to delay the expansion of G1 GC barriers until later in the compilation process.

In one sentence: Java 24's **Late Barrier Expansion for G1** keeps G1 garbage-collection barriers in an abstract representation longer during JIT compilation, allowing more optimization before finally converting those barriers into low-level machine instructions.

Java 24 — Key Derivation Function API (Preview)

Java Version: 24

Year: 2025

Feature: Key Derivation Function API (Preview)

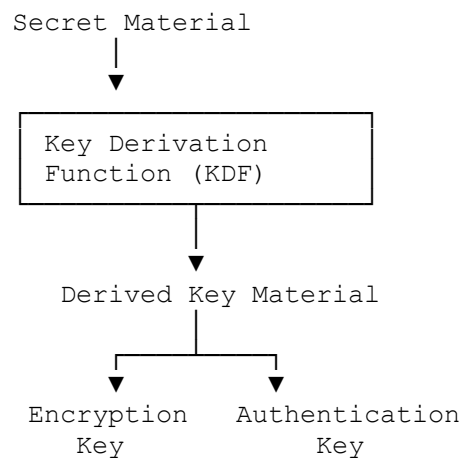
UniqueClassName: KeyDerivationFunctionAPI

Definition

The **Key Derivation Function (KDF) API** is a **preview API introduced in Java 24** that provides a standard Java interface for deriving cryptographic keys from existing secret material.

A **Key Derivation Function (KDF)** takes some initial secret material—such as a password-derived secret, shared secret, or cryptographic key—and produces one or more new cryptographic keys.

Conceptually:

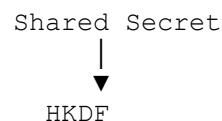


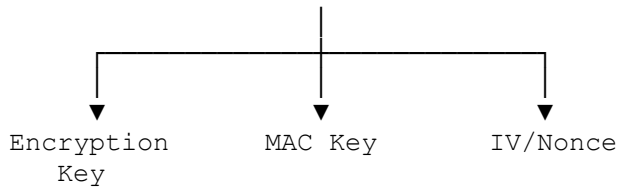
Java 24's API provides a standard abstraction around KDF algorithms, initially supporting algorithms such as **HKDF** (HMAC-based Extract-and-Expand Key Derivation Function).

Why Is a KDF Needed?

A cryptographic application often starts with one shared secret but needs **different keys for different purposes**.

For example:

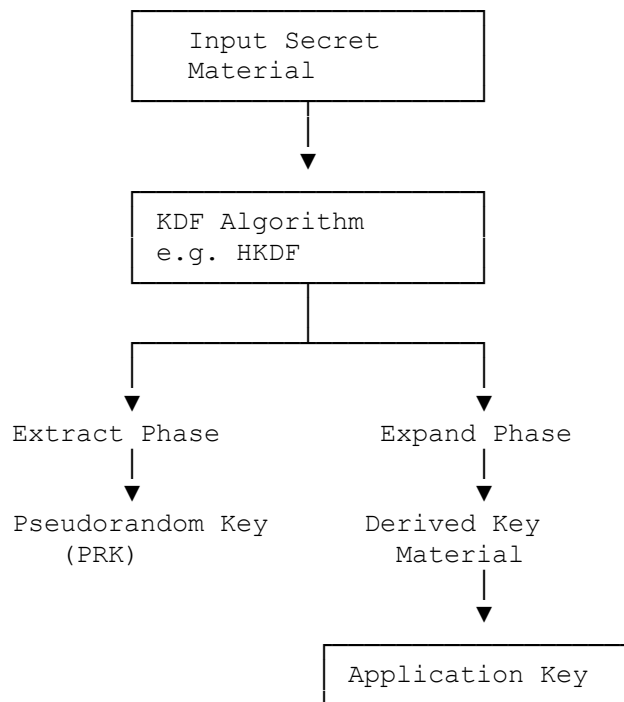




Instead of using the same secret directly everywhere, a KDF derives separate keys.

This provides **key separation** and allows an application to derive appropriately sized key material for a particular cryptographic purpose.

Key Derivation Function Flowchart



Java 24 KDF API

The API is located in:

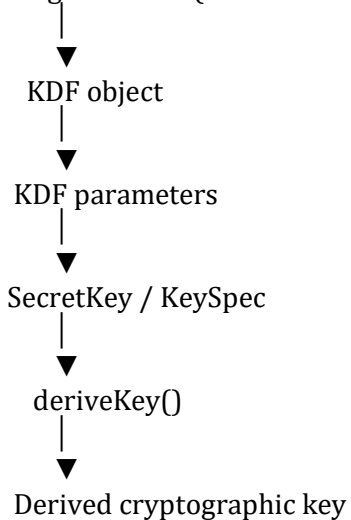
`javax.crypto`

The central class is:

`javax.crypto.KDF`

A typical workflow is:

`KDF.getInstance("HKDF-SHA256")`



Java Code Example

Because the API is a **preview feature in Java 24**, compilation and execution require preview support.

```
import javax.crypto.KDF;
import javax.crypto.SecretKey;
import javax.crypto.spec.HKDFParameterSpec;

import java.nio.charset.StandardCharsets;
import java.util.HexFormat;

public class KeyDerivationFunctionAPI {

    public static void main(String[] args) throws Exception {

        // Input secret material
```

```

byte[] inputKeyMaterial =
    "my-secret-material"
        .getBytes(StandardCharsets.UTF_8);

// Create HKDF using SHA-256
KDF kdf = KDF.getInstance("HKDF-SHA256");

// HKDF-Extract followed by HKDF-Expand
HKDFParameterSpec params =
    HKDFParameterSpec.ofExtract()
        .addIKM(inputKeyMaterial)
        .thenExpand(
            "encryption-key".getBytes(
                StandardCharsets.UTF_8),
            32
        );

// Derive a 256-bit key
SecretKey derivedKey =
    kdf.deriveKey("AES", params);

System.out.println(
    "Algorithm: " + derivedKey.getAlgorithm()
);

System.out.println(
    "Derived Key: " +
    HexFormat.of().formatHex(
        derivedKey.getEncoded()
    )
);
}
}

```

Compilation

Because this is a preview API:

```
javac --enable-preview --release 24 KeyDerivationFunctionAPI.java
```

Execution

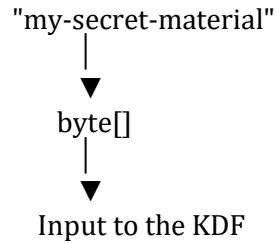
```
java --enable-preview KeyDerivationFunctionAPI
```

Understanding the Example

1. Create input key material

```
byte[] inputKeyMaterial =  
    "my-secret-material".getBytes(StandardCharsets.UTF_8);
```

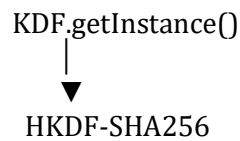
This is the initial secret material.



2. Create the KDF

```
KDF kdf = KDF.getInstance("HKDF-SHA256");
```

This requests an HKDF implementation using SHA-256.

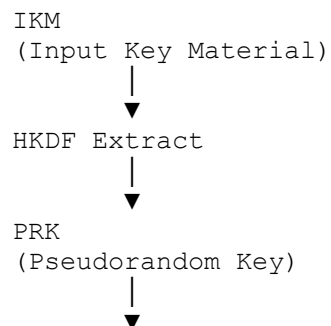


3. Supply KDF parameters

```
HKDFParameterSpec params =  
    HKDFParameterSpec.ofExtract()  
        .addIKM(inputKeyMaterial)  
        .thenExpand(  
            "encryption-key".getBytes(  
                StandardCharsets.UTF_8),  
            32  
        );
```

This describes the HKDF operation.

The important components are:



HKDF Expand
↓
info = "encryption-key"
length = 32 bytes
↓
32-byte derived key

4. Derive an AES key

```
SecretKey derivedKey =  
    kdf.deriveKey("AES", params);
```

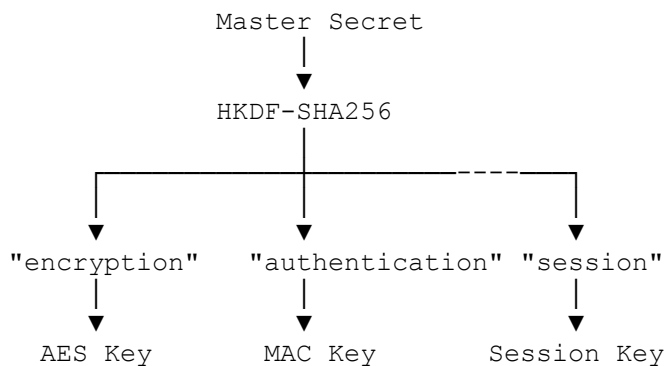
The result is a Java SecretKey that can be used with cryptographic APIs such as AES.

KDF
↓ HKDF-SHA256
SecretKey
↓ AES
Encryption / Decryption

Multiple Keys From One Secret

One of the important uses of a KDF is deriving **different keys for different purposes**.

For example:



The info value can distinguish the purpose of each derived key.

For example:

"encryption-key"

and:

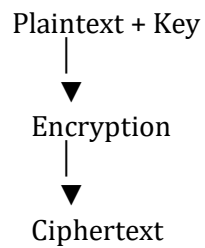
"authentication-key"

can produce different derived values from the same underlying secret.

Important Security Concept

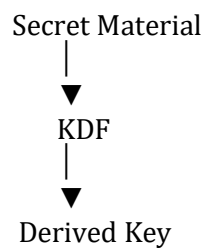
A KDF is **not the same thing as encryption**.

Encryption:



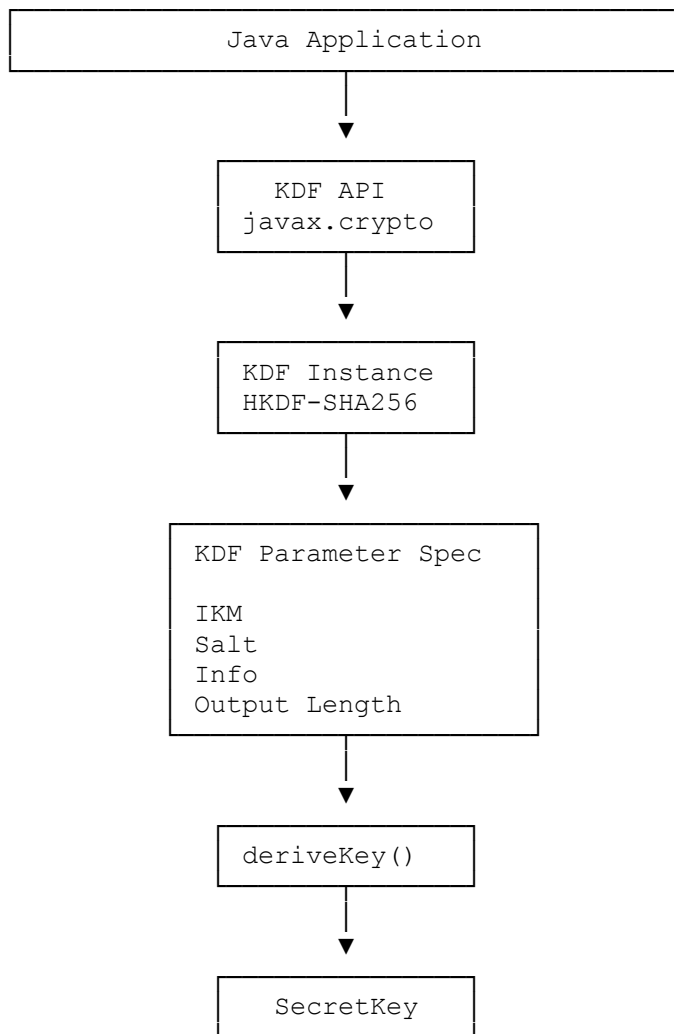
Whereas:

KDF:



The KDF produces key material; another cryptographic algorithm can then use that key.

Java 24 KDF Architecture



Key Classes

Class	Purpose
KDF	Main API for obtaining a KDF implementation and deriving keys
HKDFParameterSpec	Specifies parameters for HKDF operations
SecretKey	Represents the resulting cryptographic key
KDF.deriveKey()	Performs key derivation

Key Takeaways

- **Introduced:** Java 24
- **Year:** 2025
- **Status:** Preview
- **Package:** javax.crypto
- **Main class:** KDF
- **Initial important algorithm:** HKDF
- **Purpose:** Derive cryptographic keys from secret material
- **Java language syntax:** No new syntax
- **Main benefit:** Standardized, provider-based KDF support in the Java Cryptography Architecture
- **Typical use:** Deriving separate encryption, authentication, or session keys from shared secret material.

In one sentence: Java 24's Key Derivation Function API provides a standardized javax.crypto.KDF API for securely deriving cryptographic keys—such as AES keys—from existing secret material using algorithms such as HKDF.

Java 24 — Remove Windows 32-bit x86 Port

Java Version: 24

Year: 2025

Feature: Remove Windows 32-bit x86 Port

UniqueClassName: RemoveWindows32BitX86

Definition

Remove Windows 32-bit x86 Port is a **Java 24 platform change** in which OpenJDK no longer provides the **Windows 32-bit x86 port**.

Historically, Java supported running on both:

Windows 32-bit (x86)

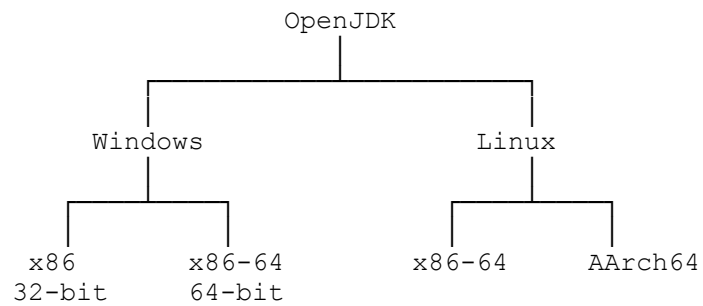
Windows 64-bit (x86-64)

Starting with **JDK 24**, the Windows x86 (32-bit) port was removed because modern Windows systems and hardware are predominantly 64-bit, while maintaining a separate 32-bit port adds development, testing, and maintenance costs.

What does "port" mean?

A **port** is a JVM implementation adapted to run on a particular operating-system and CPU architecture combination.

For example:

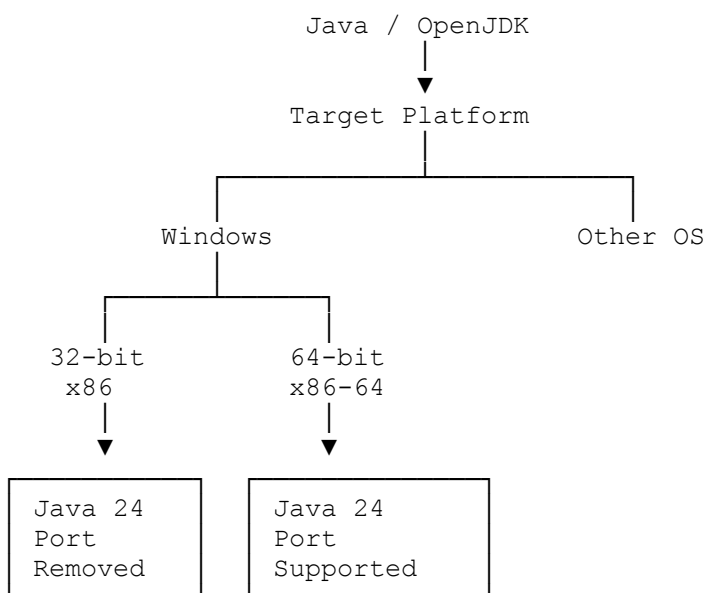


Java 24 removes the:

Windows + x86 + 32-bit

combination.

Flowchart



Before Java 24

```
Windows
├── x86 32-bit      ✓
└── x86-64 64-bit  ✓
```

Java 24

```
Windows
├── x86 32-bit      ✗ Removed
└── x86-64 64-bit  ✓
```

Why Was It Removed?

Maintaining a separate 32-bit platform requires work across many parts of the JVM:

32-bit JVM
JIT compiler Garbage collectors Native libraries OS integration Build infrastructure Testing Bug fixes Security maintenance

As 32-bit Windows usage declined, the cost of maintaining the port became increasingly difficult to justify.

The removal allows OpenJDK development to focus on actively used 64-bit platforms.

Java Code Example

This feature **does not introduce a new Java API or language syntax**.

Therefore, ordinary Java code continues to work:

```
public class RemoveWindows32BitX86 {  
    public static void main(String[] args) {  
        System.out.println("Java Version: "  
            + System.getProperty("java.version"));  
  
        System.out.println("Operating System: "  
            + System.getProperty("os.name"));  
  
        System.out.println("Architecture: "  
            + System.getProperty("os.arch"));  
  
        System.out.println("Java Architecture: "  
            + System.getProperty("sun.arch.data.model")  
            + "-bit");  
    }  
}
```

A 64-bit Windows JVM might produce output similar to:

```
Java Version      : 24  
Operating System  : Windows 11  
Architecture     : amd64  
Java Architecture : 64-bit
```

The exact values depend on the JVM and operating system.

Checking the Platform Programmatically

You can use:

```
System.getProperty("os.arch");
```

to inspect the reported architecture.

For example:

```
public class RemoveWindows32BitX86 {  
    public static void main(String[] args) {  
        String os = System.getProperty("os.name");  
        String arch = System.getProperty("os.arch");  
  
        System.out.println("OS    : " + os);  
    }  
}
```

```

System.out.println("Arch : " + arch);

if (os.startsWith("Windows")
    && arch.equalsIgnoreCase("x86")) {

    System.out.println(
        "Windows 32-bit x86 detected."
    );

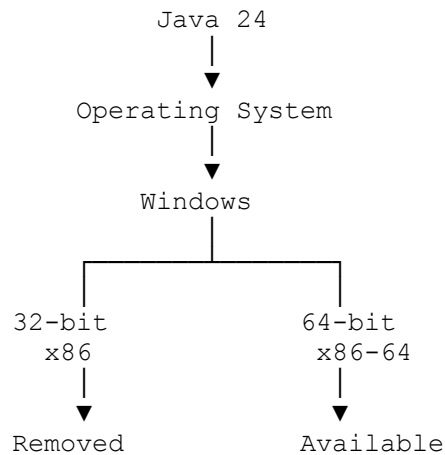
} else {
    System.out.println(
        "Running on a supported platform."
    );
}
}
}

```

However, this code **does not restore 32-bit support**. It only demonstrates how an application can inspect its runtime environment.

Platform Selection

Think of Java installation as:



If an organization has an old application running on:

Windows 32-bit
+
32-bit JDK/JRE

it needs to migrate to a supported environment, typically:

Windows 64-bit

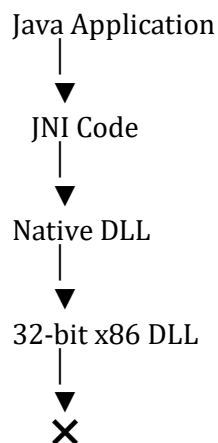
+
64-bit JDK

Impact on Java Developers

Area	Impact
Java source code	No change required
Java language	No new syntax
JVM	Windows 32-bit x86 port removed
Windows 64-bit	Continues to be supported
Existing 32-bit Windows deployments	Must migrate
Java libraries	Generally unaffected at source level
Native/JNI applications	Need particular attention to native binaries
New applications	Should target supported 64-bit platforms

Special consideration: JNI

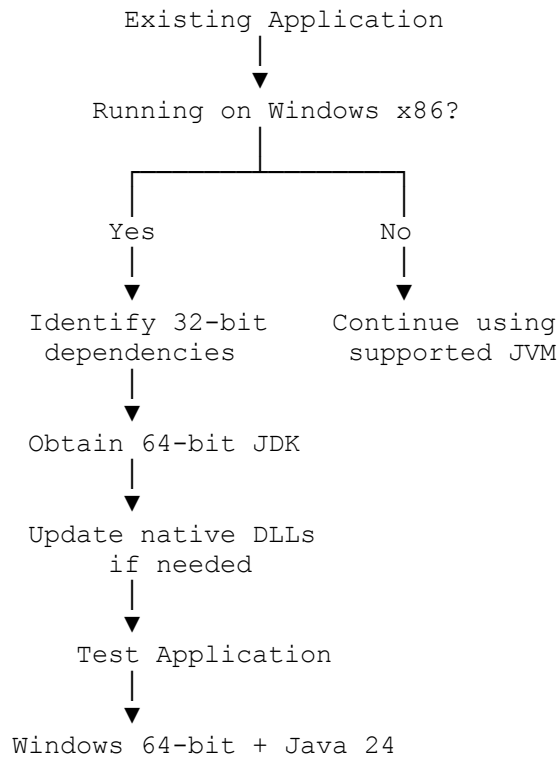
Applications using native libraries should check their native dependencies carefully:



A 32-bit native DLL cannot simply be loaded into a 64-bit JVM.

The migration may therefore require obtaining or rebuilding **64-bit native libraries**.

Overall Migration Flow



Key Takeaways

- **Introduced:** Java 24
- **Year:** 2025
- **Type:** Platform/JVM port removal
- **Removed:** Windows 32-bit x86 port
- **Not removed:** Windows 64-bit x86-64 support
- **Java source changes:** Normally none
- **Main reason:** Declining relevance of 32-bit Windows and reduced maintenance burden
- **Migration target:** Windows 64-bit with a 64-bit JDK
- **Extra concern:** Applications using 32-bit JNI/native DLLs may need native-library migration.

In one sentence: Java 24 removes OpenJDK's Windows 32-bit x86 port, allowing the JDK to focus on modern 64-bit Windows environments while applications running on 32-bit Windows must migrate to a supported 64-bit platform.

Java 24 — Ahead-of-Time Class Loading & Linking

Java Version: 24

Year: 2025

Feature: Ahead-of-Time Class Loading & Linking

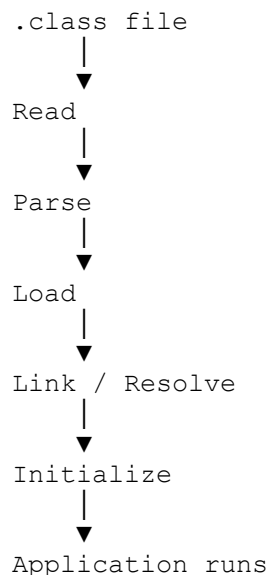
JEP: 483

UniqueClassName: AOTClassLoadingLinking

Definition

Ahead-of-Time (AOT) Class Loading & Linking is a Java 24 JVM feature that improves **application startup and warm-up performance** by preparing classes **before the application starts** and storing their loaded and linked state in an **AOT cache**. JEP 483 is part of the Project Leyden effort to reduce Java startup and warm-up costs. ([OpenJDK Bug Tracker](#))

Normally, when a JVM starts an application, it must perform operations such as:



With AOT class loading and linking, the JVM can perform much of the **loading and linking work ahead of time**, save the resulting information in an AOT cache, and reuse it during later application launches. ([OpenJDK Bug Tracker](#))

Main Goal

Traditional Startup

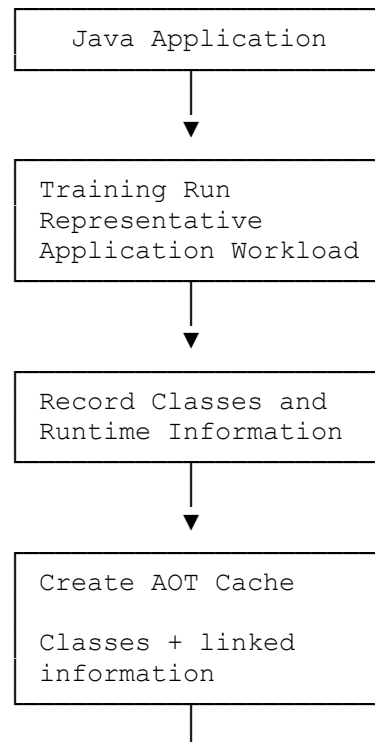
Application

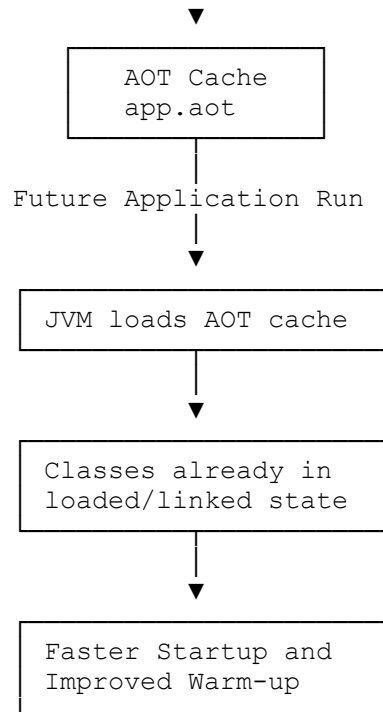
↓
Load classes
↓
Link classes
↓
Resolve classes
↓
Start application

AOT Startup

Training run
↓
Record classes
↓
Create AOT cache
↓
Future application start
↓
Reuse loaded/linked classes
↓
Start faster

Flowchart





Three-Phase AOT Process

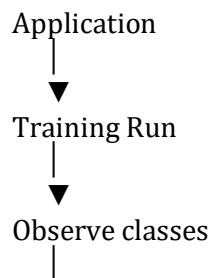
JDK 24's AOT workflow can be understood as three major phases. OpenJDK documents the sequence as **training** → **cache creation** → **production use**. ([OpenJDK Bug Tracker](#))

1. Training

Run the application with a representative workload:

```
java -XX:AOTMode=record \  
-XX:AOTConfiguration=app.aotconfig \  
-cp app.jar \  
com.example.Main
```

The JVM records information about the classes used by the application.



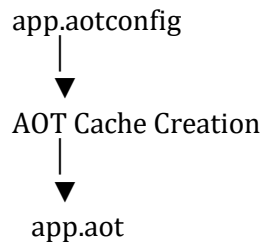
▼
app.aotconfig

2. Create the AOT Cache

Create the cache:

```
java -XX:AOTMode=create \  
      -XX:AOTConfiguration=app.aotconfig \  
      -XX:AOTCache=app.aot \  
      -cp app.jar
```

The result is an AOT cache containing reusable AOT artifacts.

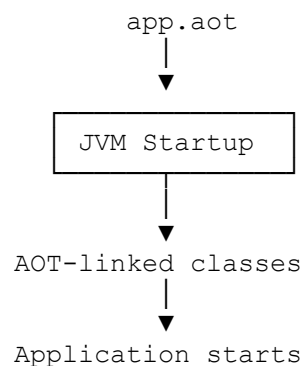


3. Production Run

Run the application using the cache:

```
java -XX:AOTCache=app.aot \  
      -cp app.jar \  
      com.example.Main
```

Conceptually:



The official JEP describes the objective as making application classes available in a loaded and linked state when the JVM starts. ([OpenJDK Bug Tracker](#))

Java Code Example

AOT Class Loading & Linking **does not require special Java language syntax**. Your Java application can be completely ordinary.

```
public class AOTClassLoadingLinking {

    static class Customer {

        private final int id;
        private final String name;

        Customer(int id, String name) {
            this.id = id;
            this.name = name;
        }

        void display() {
            System.out.println(
                "Customer: " + id + " - " + name
            );
        }
    }

    public static void main(String[] args) {

        System.out.println("Application starting...");

        Customer customer =
            new Customer(101, "Anurag");

        customer.display();

        System.out.println("Application completed.");
    }
}
```

Compile:

```
javac AOTClassLoadingLinking.java
```

Package it into a JAR:

```
jar --create \
```

```
--file app.jar \  
--main-class AOTClassLoadingLinking \  
AOTClassLoadingLinking.class
```

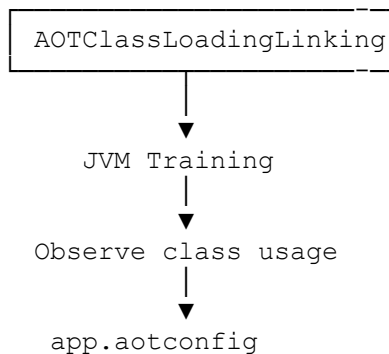
The AOT mechanism operates at the **JVM/runtime level**, not through changes to the Customer class or main() method.

Complete AOT Example

Step 1 — Training Run

```
java -XX:AOTMode=record \  
-XX:AOTConfiguration=app.aotconfig \  
-cp app.jar \  
AOTClassLoadingLinking
```

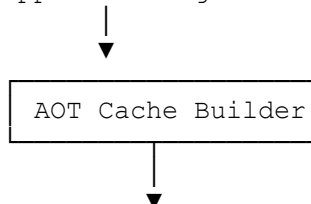
Conceptually:



Step 2 — Create Cache

```
java -XX:AOTMode=create \  
-XX:AOTConfiguration=app.aotconfig \  
-XX:AOTCache=app.aot \  
-cp app.jar
```

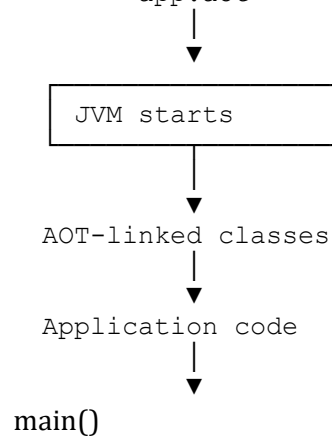
app.aotconfig



app.aot

Step 3 — Run With Cache

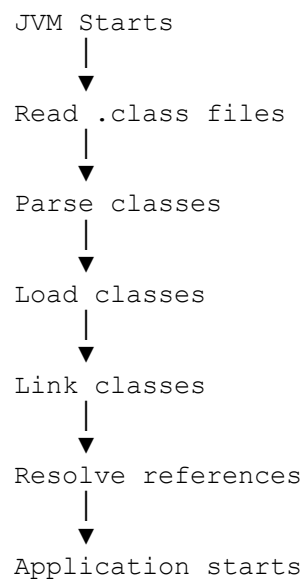
```
java -XX:AOTCache=app.aot \  
-cp app.jar \  
AOTClassLoadingLinking  
app.aot
```



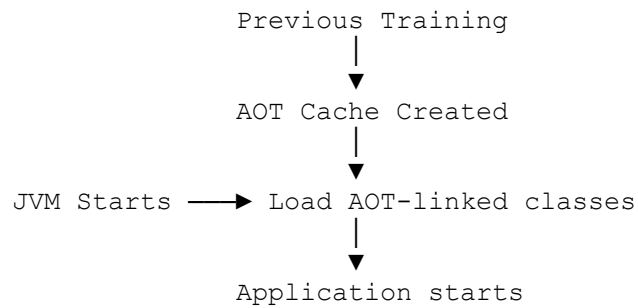
These AOT command-line modes are documented as the JDK 24 workflow for JEP 483.

Traditional vs AOT Startup

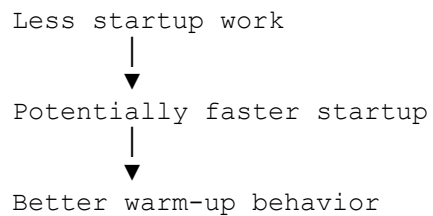
Traditional



AOT



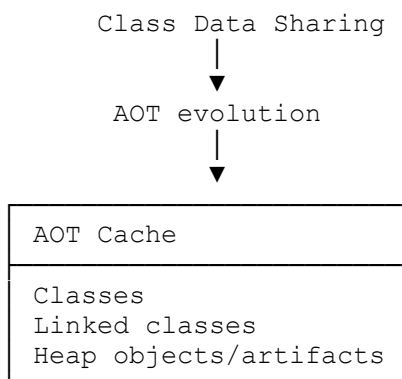
Therefore:



OpenJDK performance testing for JEP 483 specifically targeted startup improvements, with larger expected benefits for applications making significant use of invokedynamic; the reported benchmark expectations included roughly 15–30% for a stream test and 10–20% for javac in the tested scenarios. These are **benchmark-specific expectations, not a universal performance guarantee**. ([OpenJDK Bug Tracker](#))

Relationship With CDS

AOT Class Loading & Linking builds upon the JVM's existing **Class Data Sharing (CDS)** technology.



The JDK 24 AOT cache is described as a natural evolution of CDS and can store classes and heap objects; future releases may extend the cache to other AOT artifacts. ([OpenJDK](#))

AOTClassLinking

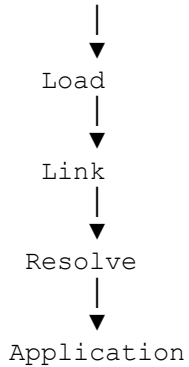
A particularly important JVM option is:

`-XX:+AOTClassLinking`

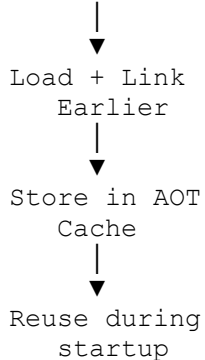
When enabled during AOT cache creation, the JVM performs more advanced class-linking optimizations, including ahead-of-time resolution of certain invokedynamic operations. This can further improve startup and warm-up performance, subject to compatibility restrictions. ([OpenJDK](#))

Conceptually:

Normal Class Loading



AOT Class Linking



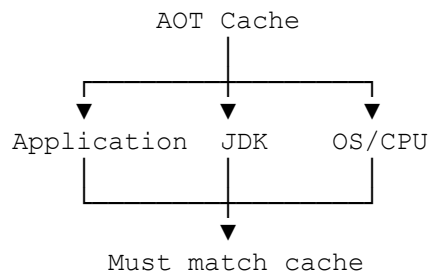
In JDK 24, when `-XX:AOTMode` is used, AOT class linking is automatically enabled; it can be explicitly disabled with `-XX:-AOTClassLinking`. ([OpenJDK](#))

Important Limitations

An AOT cache is associated with a particular:

- application/class path or module path,
- JDK release,
- operating system,
- CPU architecture.

If these change, the cache needs to be recreated.



Key Takeaways

Aspect	Details
Java Version	24
Year	2025
JEP	483
Feature	Ahead-of-Time Class Loading & Linking
Area	HotSpot / Runtime
Main goal	Improve startup and warm-up
Technology	AOT Cache + CDS infrastructure
Java syntax changes	None
Training	<code>-XX:AOTMode=record</code>

Cache creation	-XX:AOTMode=create
Cache usage	-XX:AOTCache=app.aot
Advanced linking	-XX:+AOTClassLinking
Main benefit	Reduce class-loading/linking work during startup

One-line definition

Java 24's Ahead-of-Time Class Loading & Linking (JEP 483) records an application's class usage, stores loaded and linked classes in an AOT cache, and reuses them during subsequent JVM launches to reduce startup and warm-up overhead. ([OpenJDK Bug Tracker](#))

Java 24 — Class-File API

Java Version: 24

Year: 2025

Feature: Class-File API

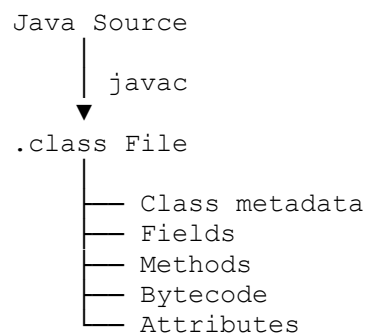
JEP: 484

UniqueClassName: ClassFileAPI

Definition

The **Class-File API** is a **final API in Java 24** for reading, writing, and transforming Java **class files (.class)**.

A Java class file is the binary representation of a compiled Java class:



The Class-File API provides a modern, standard replacement for libraries that previously had to manipulate class files using internal JDK APIs or third-party bytecode libraries.

The API is primarily located in:

`java.lang.classfile`

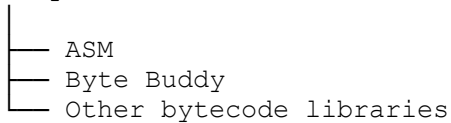
It provides APIs for:

- Reading class files
 - Parsing class-file structures
 - Creating class files
 - Transforming class files
 - Generating bytecode
 - Working with fields and methods
 - Inspecting instructions and attributes
-

Why Was the Class-File API Introduced?

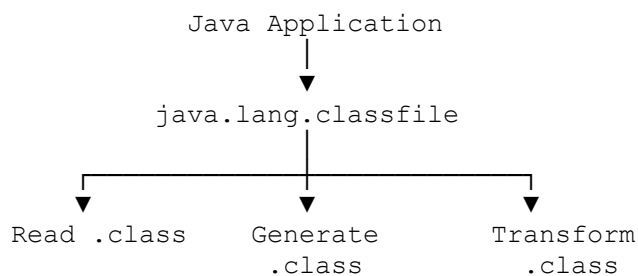
Before this API, developers commonly used:

Third-party libraries



or internal JDK APIs.

Java 24 provides a standardized API:

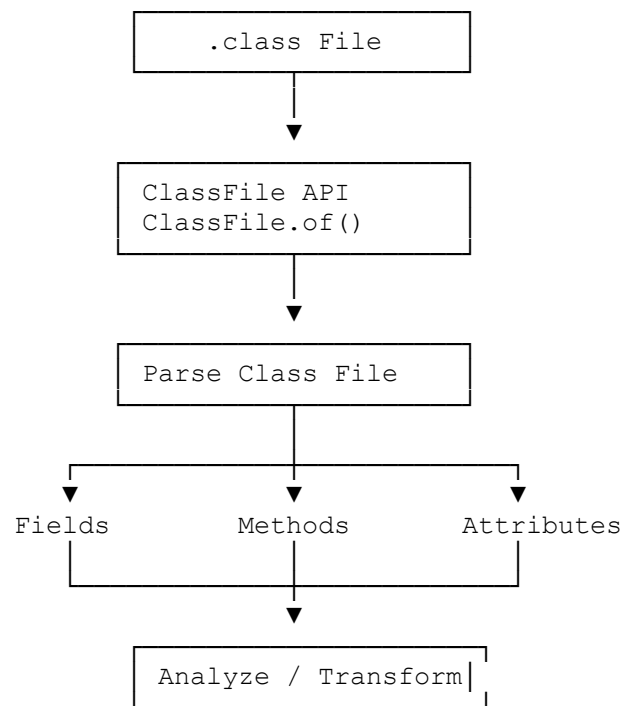


This is particularly useful for:

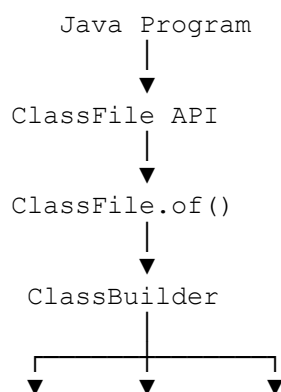
- Frameworks
- Compilers
- Build tools

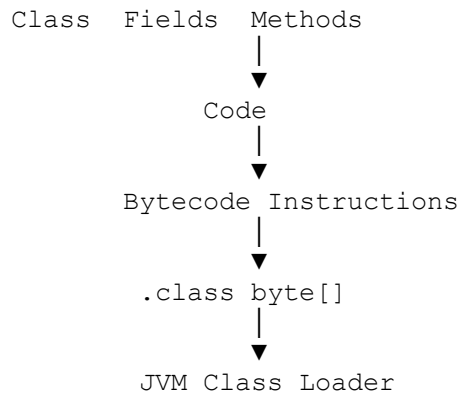
- JVM language implementations
 - Bytecode instrumentation
 - Code-generation tools
 - Static-analysis tools
 - Runtime agents
-

Flowchart — Reading a Class File



Flowchart — Generating a Class





Main API Types

Some important types in `java.lang.classfile` include:

API	Purpose
ClassFile	Main entry point for reading, parsing, generating, and transforming class files
ClassModel	Represents a parsed class file
ClassTransform	Defines transformations applied to class-file elements
MethodModel	Represents a method in a class file
FieldModel	Represents a field
CodeModel	Represents method bytecode
ClassBuilder	Used to construct a new class file
CodeBuilder	Used to construct method bytecode

Java Code Example 1 — Reading a .class File

Suppose we have:

```
public class Student {  
  
    private int id;  
  
    public Student(int id) {  
        this.id = id;  
    }  
  
    public int getId() {  
        return id;  
    }  
}
```

After compilation:

Student.java



Student.class

We can inspect the class file using the Class-File API.

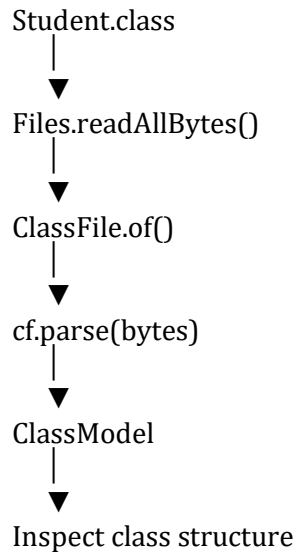
```
import java.lang.classfile.ClassFile;  
  
import java.nio.file.Files;  
import java.nio.file.Path;  
  
public class ClassFileAPI {  
  
    public static void main(String[] args) throws Exception {  
  
        Path classFile =  
            Path.of("Student.class");  
  
        byte[] bytes =  
            Files.readAllBytes(classFile);  
  
        ClassFile cf =  
            ClassFile.of();  
  
        var model =  
            cf.parse(bytes);  
  
        System.out.println(  
            "Class: " + model.thisClass().asSymbol()  
        );  
    }  
}
```

```

    );
    model.forEach(component ->
        System.out.println(component)
    );
}
}

```

The basic flow is:



Java Code Example 2 — Generating a Class

The Class-File API can also create class-file bytes.

For example, we can generate a class named:

GeneratedClass

with a simple main() method.

```

import java.lang.classfile.ClassFile;
import java.lang.classfile.attribute.ConstantValueAttr;

import java.nio.file.Files;
import java.nio.file.Path;

public class ClassFileAPI {

    public static void main(String[] args)

```

```

        throws Exception {

        ClassFile cf = ClassFile.of();

        byte[] classBytes =
            cf.build(
                ClassFileAPI.class.describeConstable()
                    .orElseThrow(),
                builder -> {
                    // Class-file construction
                }
            );

        Files.write(
            Path.of("GeneratedClass.class"),
            classBytes
        );

        System.out.println(
            "Class file generated."
        );
    }
}

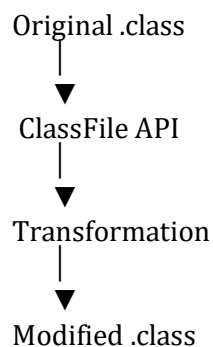
```

For practical bytecode generation, the `ClassBuilder` and `CodeBuilder` APIs are used to define the class structure and bytecode instructions.

Java Code Example 3 — Transforming a Class

One of the most useful capabilities is **class-file transformation**.

Conceptually:



For example:

```

import java.lang.classfile.ClassFile;
import java.lang.classfile.ClassTransform;

```

```

import java.nio.file.Files;
import java.nio.file.Path;

public class ClassFileAPI {

    public static void main(String[] args)
        throws Exception {

        Path input =
            Path.of("Student.class");

        Path output =
            Path.of("StudentModified.class");

        byte[] original =
            Files.readAllBytes(input);

        ClassFile cf =
            ClassFile.of();

        byte[] transformed =
            cf.transform(
                original,
                ClassTransform.ACCEPT_ALL
            );

        Files.write(output, transformed);

        System.out.println(
            "Class transformation completed."
        );
    }
}

```

ACCEPT_ALL effectively represents a transformation that accepts the existing class-file elements without modifying them. Real transformations can replace, remove, or add class-file elements.

Bytecode Transformation Concept

Consider:

```

public int add(int a, int b) {
    return a + b;
}

```

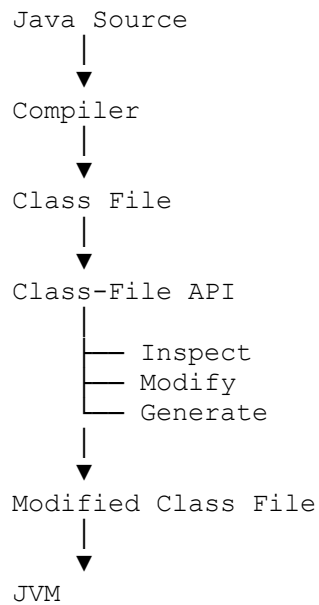
At the source level:

a + b

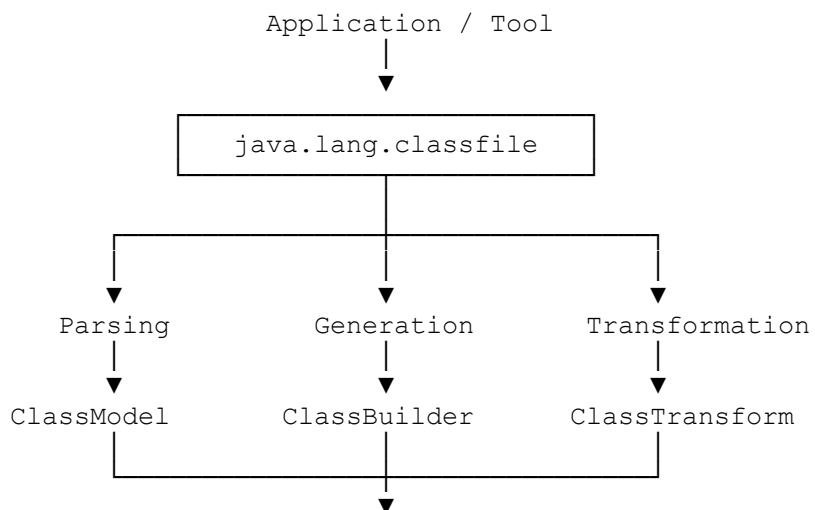
becomes JVM bytecode conceptually similar to:

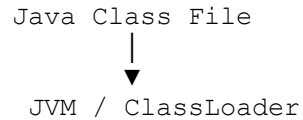
```
iload a
iload b
iadd
ireturn
```

The Class-File API lets tools work at this bytecode/class-file level.



Class-File API Architecture

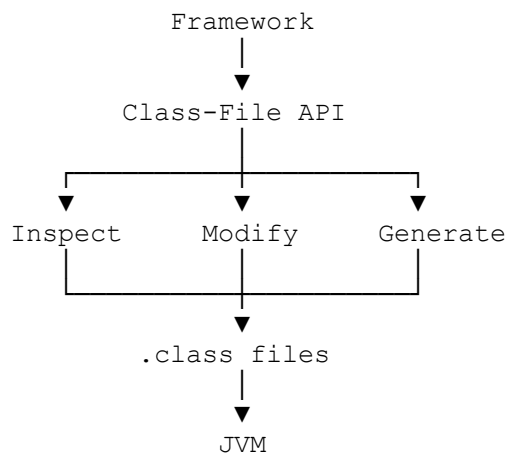




Example Use Case — Frameworks

Frameworks often generate or modify classes.

For example:



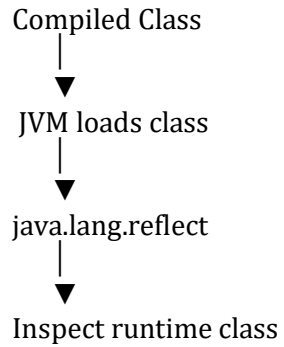
This can be useful for:

- Dependency injection frameworks
 - ORM frameworks
 - Serialization frameworks
 - Mocking frameworks
 - Profilers
 - Code coverage tools
 - Java agents
 - Bytecode optimizers
 - Dynamic language implementations
-

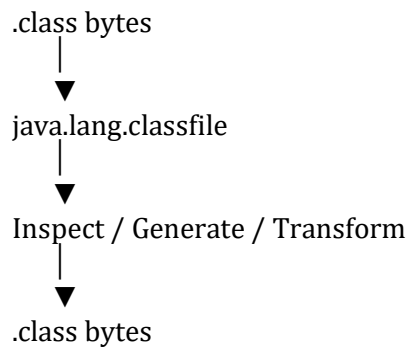
Class-File API vs Java Reflection

These APIs operate at different levels.

Reflection



Class-File API

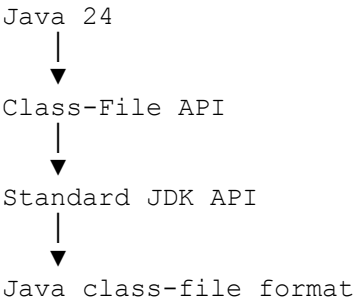


Therefore:

Feature	Reflection	Class-File API
Works with	Loaded classes	Class-file representation
Inspect methods	✓	✓
Inspect fields	✓	✓
Generate classes	Limited	✓
Modify bytecode	✗	✓
Transform class files	✗	✓
Requires loading class	Generally yes	No
Package	java.lang.reflect	java.lang.classfile

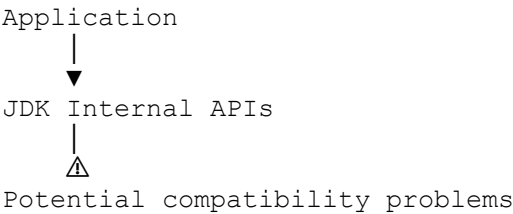
Important Advantage

The Class-File API is designed to track changes in the **Java class-file format** across Java releases.

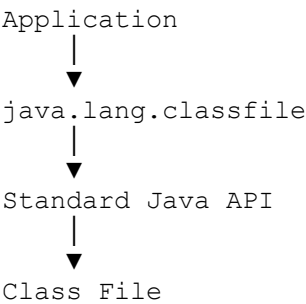


This is especially useful because class-file formats evolve as Java introduces new JVM features.

Instead of relying on internal APIs:



developers can use:



Key Takeaways

Property	Details
Java Version	24
Year	2025

JEP	484
Status	Final
Package	java.lang.classfile
Purpose	Read, write, generate, and transform class files
Java syntax changes	None
Main entry point	ClassFile
Parsing representation	ClassModel
Transformation	ClassTransform
Generation	ClassBuilder
Bytecode generation	CodeBuilder
Useful for	Frameworks, agents, compilers, instrumentation, code generation

One-line definition

Java 24's Class-File API (JEP 484) is a standard java.lang.classfile API for parsing, generating, inspecting, and transforming Java class files without relying on internal JDK APIs or requiring third-party bytecode libraries.

Java 24 — Stream Gatherers

Java Version: 24

Year: 2025

Feature: Stream Gatherers

JEP: 485

UniqueClassName: StreamGatherers

Definition

Stream Gatherers are a **final feature in Java 24** that provide a mechanism for defining **custom intermediate operations** for the Java Stream API.

Before Stream Gatherers, Java Streams provided many built-in intermediate operations such as:

```
map()
filter()
flatMap()
distinct()
sorted()
limit()
```

But developers could not easily create their own reusable intermediate stream operation.

Gatherers solve this problem.

They allow you to define custom operations that can:

- Transform elements
- Maintain state
- Process elements in groups
- Produce a different number of output elements
- Perform windowing
- Implement custom aggregation-like stream behavior
- Potentially process streams in parallel

The API is in:

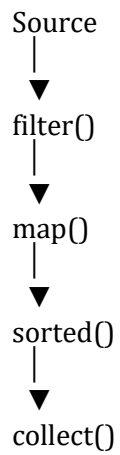
```
java.util.stream.Gatherer
```

and is used through:

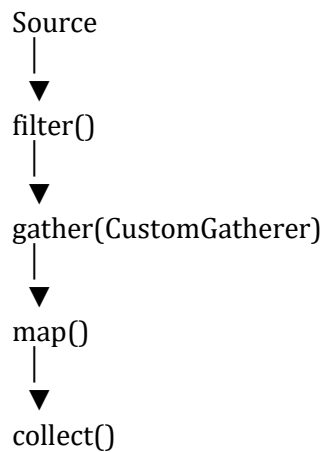
```
Stream.gather(...)
```

Basic Concept

A normal Stream pipeline looks like:

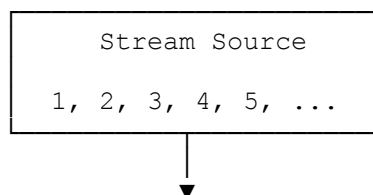


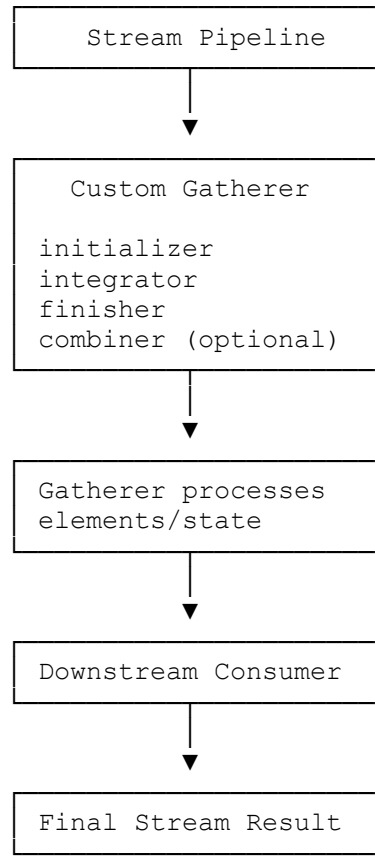
With a Gatherer:



The key difference is that **you can define what happens inside the custom intermediate operation.**

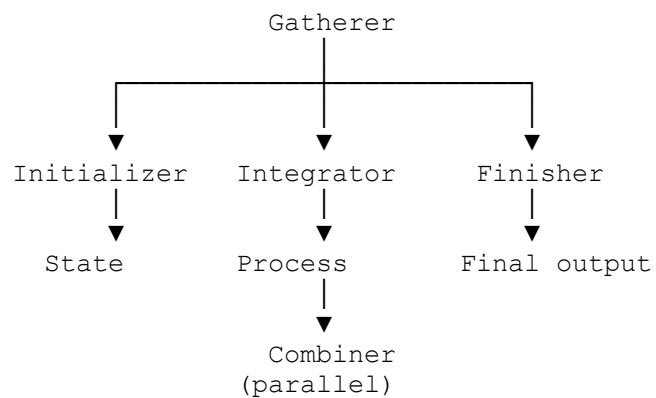
Flowchart





Gatherer Architecture

A Gatherer can be understood as four major components:

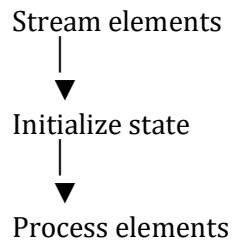


1. Initializer

Creates the state used while processing the stream.

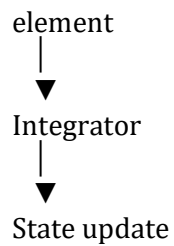
Gatherer.Integrator

conceptually:



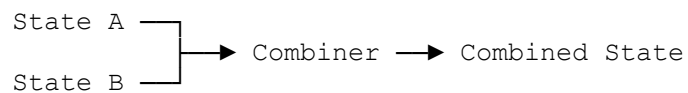
2. Integrator

Receives input elements one at a time and updates the state.



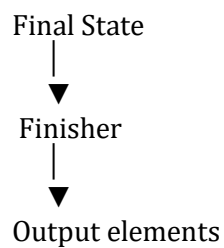
3. Combiner

Combines states when processing occurs in parallel.



4. Finisher

Produces final output when the upstream stream is finished.



Java Code Example 1 — Simple Custom Gatherer

Let's create a gatherer that converts every input number into its **square**.

```
import java.util.stream.Gatherer;
import java.util.stream.Stream;

public class StreamGatherers {

    public static void main(String[] args) {

        Stream<Integer> numbers =
            Stream.of(1, 2, 3, 4, 5);

        var result =
            numbers.gather(
                Gatherer.of(
                    (downstream, element) -> {

                        downstream.push(
                            element * element
                        );

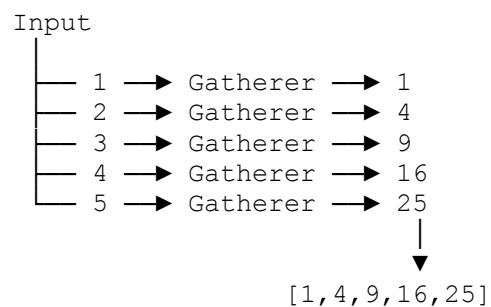
                        return true;
                    }
                )
            )
            .toList();

        System.out.println(result);
    }
}
```

Output:

[1, 4, 9, 16, 25]

Flow



For this simple transformation, `map()` would normally be preferable:

```
var result =  
    Stream.of(1, 2, 3, 4, 5)  
        .map(n -> n * n)  
        .toList();
```

The point of the example is to demonstrate the **Gatherer mechanism**.

Java Code Example 2 — Stateful Gatherer

The real strength of Gatherers appears when an operation needs **state**.

For example, suppose we want to calculate the running sum:

```
Input:  1  2  3  4  5  
Output: 1  3  6 10 15
```

A Gatherer can maintain the current sum.

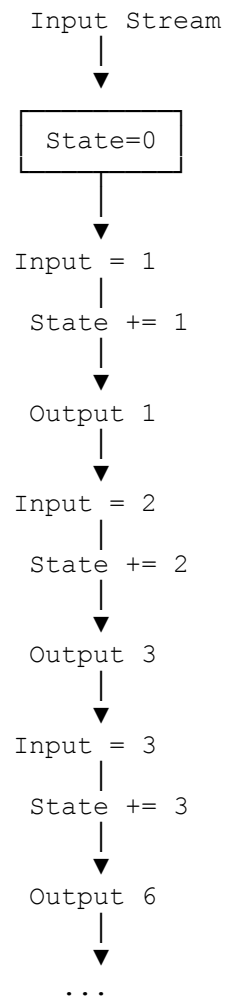
```
import java.util.stream.Gatherer;  
import java.util.stream.Stream;  
  
public class StreamGatherers {  
  
    public static void main(String[] args) {  
  
        Gatherer<Integer, int[], Integer> runningTotal =  
            Gatherer.ofSequential(  
                () -> new int[]{0},  
  
                (state, element, downstream) -> {  
  
                    state[0] += element;  
  
                    downstream.push(state[0]);  
  
                    return true;  
                }  
            );  
  
        var result =  
            Stream.of(1, 2, 3, 4, 5)  
                .gather(runningTotal)  
                .toList();  
  
        System.out.println(result);  
    }  
}
```

```
}  
}
```

Output:

[1, 3, 6, 10, 15]

Flowchart



Java Code Example 3 — Fixed-Size Windowing

One of the most useful applications of Gatherers is **windowing**.

Suppose the input is:

1 2 3 4 5 6 7 8 9

We want groups of three:

[1,2,3]

[4,5,6]

[7,8,9]

Java 24 provides predefined gatherers that make common patterns easier.

For example:

```
import java.util.stream.Gatherers;
import java.util.stream.Stream;

public class StreamGatherers {

    public static void main(String[] args) {

        var result =
            Stream.of(
                1, 2, 3,
                4, 5, 6,
                7, 8, 9
            )
            .gather(
                Gatherers.windowFixed(3)
            )
            .toList();

        System.out.println(result);
    }
}
```

Output:

```
[[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

Sliding Window

Gatherers also provide a sliding-window operation.

```
import java.util.stream.Gatherers;
import java.util.stream.Stream;

public class StreamGatherers {

    public static void main(String[] args) {
```

```

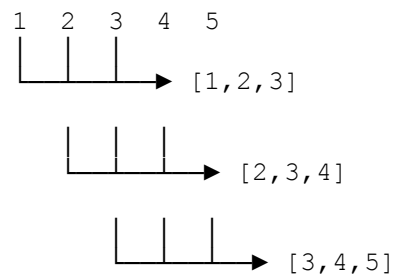
var result =
    Stream.of(1, 2, 3, 4, 5)
        .gather(
            Gatherers.windowSliding(3)
        )
        .toList();

System.out.println(result);
}
}

```

Conceptually:

Input:



Output:

```

[ [1, 2, 3],
  [2, 3, 4],
  [3, 4, 5] ]

```

Built-in Gatherers

Java 24 includes predefined gatherers in:

`java.util.stream.Gatherers`

Some important examples include:

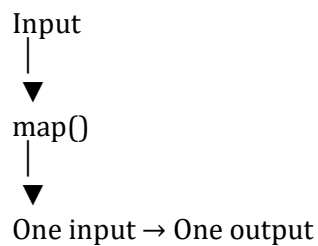
Gatherer	Purpose
windowFixed(n)	Groups elements into fixed-size windows
windowSliding(n)	Creates overlapping sliding windows
fold(...)	Performs stateful folding

scan(...)	Produces intermediate accumulated results
mapConcurrent(...)	Performs concurrent mapping

These predefined operations are useful when you don't need to write a custom Gatherer.

Gatherer vs map()

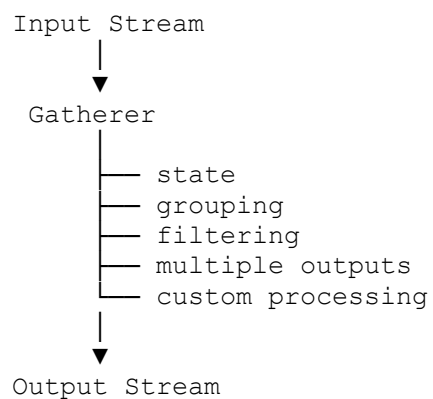
map()



Example:

```
stream.map(x -> x * 2);
```

Gatherer



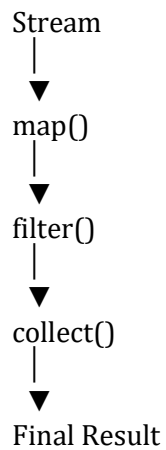
A gatherer can implement much more complex intermediate behavior.

Gatherer vs Collector

These two concepts are easy to confuse.

Collector

A **Collector** is generally used to terminate a stream:

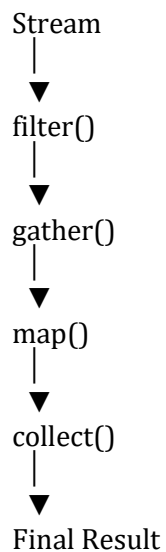


For example:

```
List<Integer> result =  
    stream.collect(Collectors.toList());
```

Gatherer

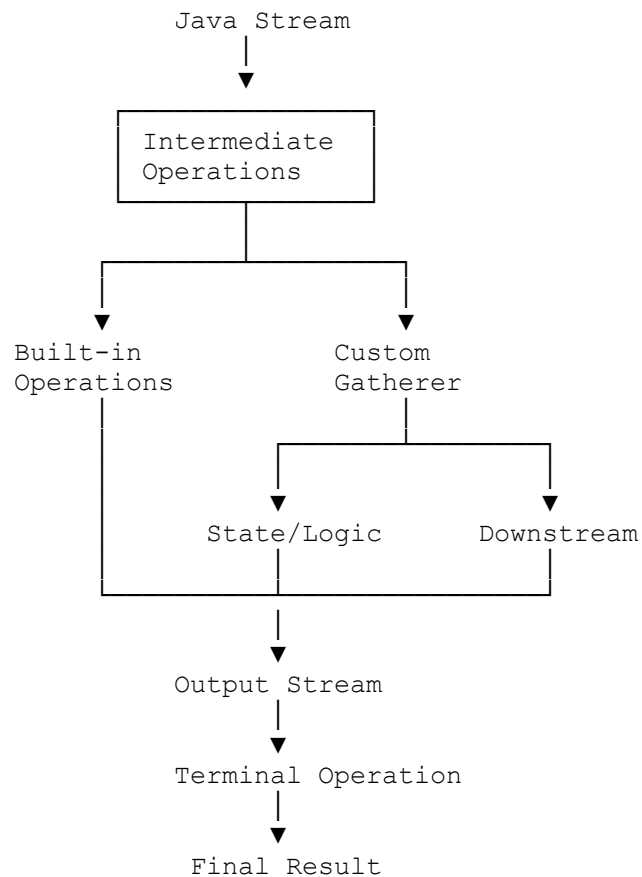
A **Gatherer** is an intermediate stream operation:



Therefore:

Collector → Terminal operation
Gatherer → Intermediate operation

Overall Architecture

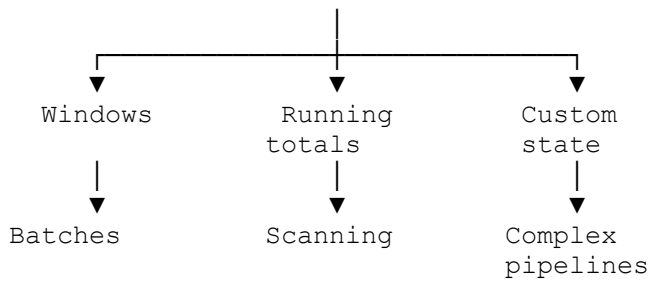


When Should You Use Stream Gatherers?

Gatherers are particularly useful when a stream operation needs **state or custom control over how input elements become output elements**.

Examples:

Stream Gatherers



Typical applications include:

- Fixed-size batches
- Sliding windows
- Running totals
- Stateful transformations
- Custom stream algorithms
- Event stream processing
- Grouping adjacent elements
- Custom filtering/transformation logic

Key Takeaways

Property	Details
Java Version	24
Year	2025
JEP	485
Status	Final
Package	java.util.stream
Main interface	Gatherer
Built-in gatherers	Gatherers
Stream method	Stream.gather()
Type	Intermediate stream operation
Supports state	Yes
Supports custom operations	Yes
Supports windowing	Yes

Can work with parallel streams	Yes, when the gatherer supports it
--------------------------------	------------------------------------

One-line definition

Java 24 Stream Gatherers (JEP 485) provide a standard way to create custom, stateful intermediate Stream operations that can transform, group, accumulate, or otherwise control how stream elements flow through a pipeline.

Java 24 — Permanently Disable Security Manager

Java Version: 24

Year: 2025

Feature: Permanently Disable Security Manager

JEP: 486

UniqueClassName: DisableSecurityManager

Definition

Permanently Disable Security Manager is a **Java 24 platform change** that permanently disables the **Security Manager**.

The Security Manager was an older Java security mechanism that allowed applications to restrict potentially dangerous operations, such as:

- Reading/writing files
- Opening network connections
- Loading native libraries
- Creating processes
- Accessing certain system properties

Historically, an application could install a security manager:

```
System.setSecurityManager(new SecurityManager());
```

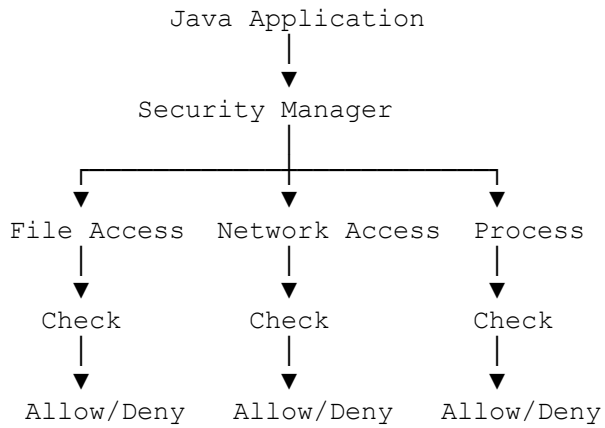
However, the Security Manager had been **deprecated for removal in Java 17**. Java 24 completes that process by permanently disabling it.

The important consequence is:

Java 24 no longer allows the Security Manager to be enabled.

What Was the Security Manager?

The historical architecture looked like this:



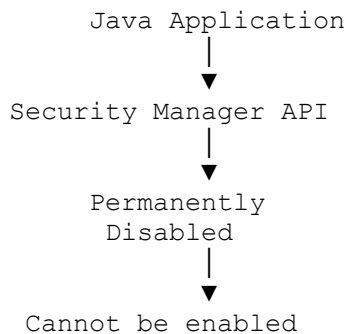
For example:

```
System.setSecurityManager(  
    new SecurityManager()  
);
```

could install a Security Manager that performed security checks.

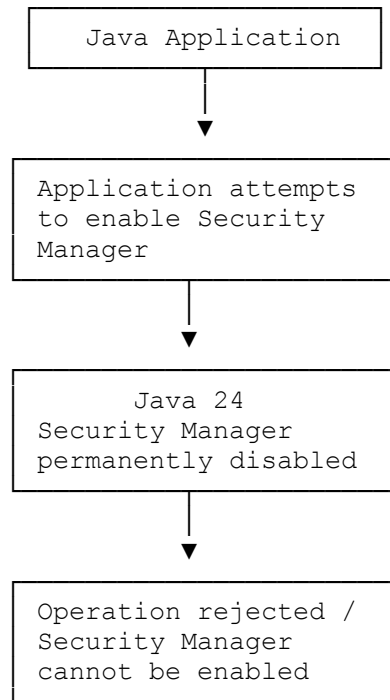
Java 24 Architecture

In Java 24, that model is no longer available:



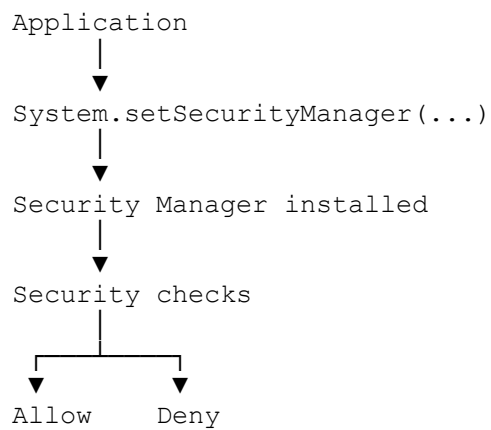
Instead of relying on the Security Manager, applications should use **modern operating-system, container, JVM, and application-level security mechanisms** appropriate to their deployment environment.

Flowchart



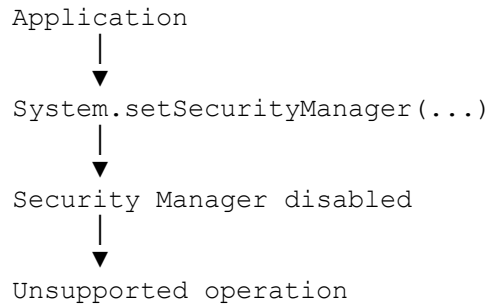
Before Java 24

Historically:



The Security Manager could intercept security-sensitive operations.

Java 24



The Security Manager is no longer a usable sandboxing mechanism.

Java Code Example

The following code demonstrates the **old approach**:

```
public class DisableSecurityManager {  
    public static void main(String[] args) {  
        System.setSecurityManager(  
            new SecurityManager()  
        );  
        System.out.println(  
            "Security Manager installed."  
        );  
    }  
}
```

Historical behavior

On older Java releases where the Security Manager was available, this attempted to install one.

In Java 17 and later, the operation was already subject to the deprecation/removal transition and warnings/restrictions.

In **Java 24**, the Security Manager is permanently disabled, so this approach no longer works.

Checking the Security Manager

Older code could do:

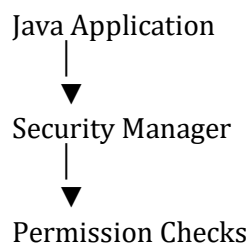
```
SecurityManager manager =  
    System.getSecurityManager();  
  
if (manager != null) {  
    System.out.println(  
        "Security Manager is active."  
    );  
}
```

This is also part of the API that should no longer be relied upon in Java 24.

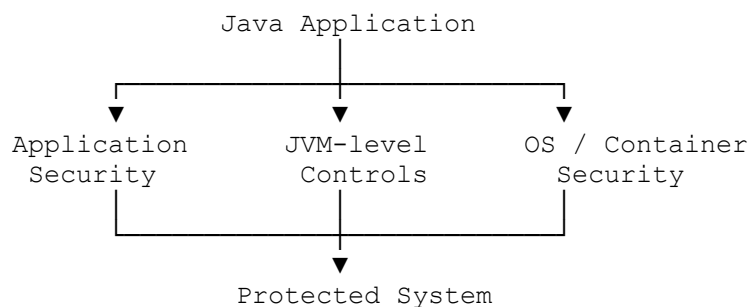
Modern Java applications should **not design security architecture around SecurityManager**.

A Modern Security Approach

Instead of:



a modern deployment might use multiple security layers:



For example:

Application authorization
Authentication / RBAC
Container / sandbox policy
Operating-system permissions
Filesystem/network controls

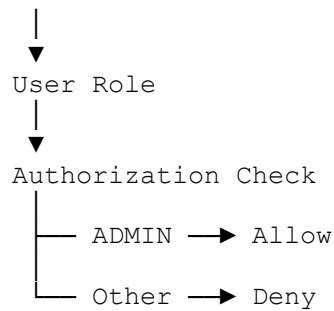
Example: Application-Level Authorization

Instead of using a Security Manager to decide whether a user can perform an operation, an application can explicitly authorize the operation:

```
public class DisableSecurityManager {  
    static boolean canDeleteCustomer(String role) {  
        return "ADMIN".equals(role);  
    }  
  
    public static void main(String[] args) {  
        String userRole = "ADMIN";  
  
        if (canDeleteCustomer(userRole)) {  
            System.out.println(  
                "Customer deletion authorized."  
            );  
        } else {  
            System.out.println(  
                "Access denied."  
            );  
        }  
    }  
}
```

Flow:

```
User  
|  
▼  
Authentication
```



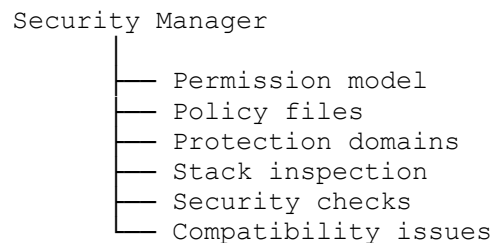
This is **application-level authorization**, not a replacement Security Manager API.

Why Was the Security Manager Removed?

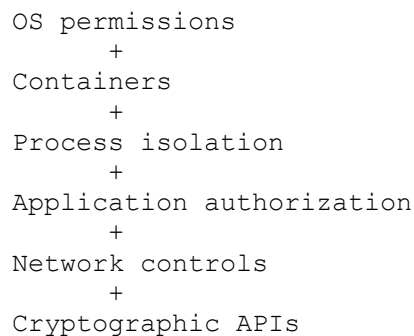
The Security Manager was originally designed when Java applications were commonly expected to run potentially untrusted code inside the JVM.

Over time, the Java security model and deployment environment changed.

The older model became difficult to maintain and had significant complexity:



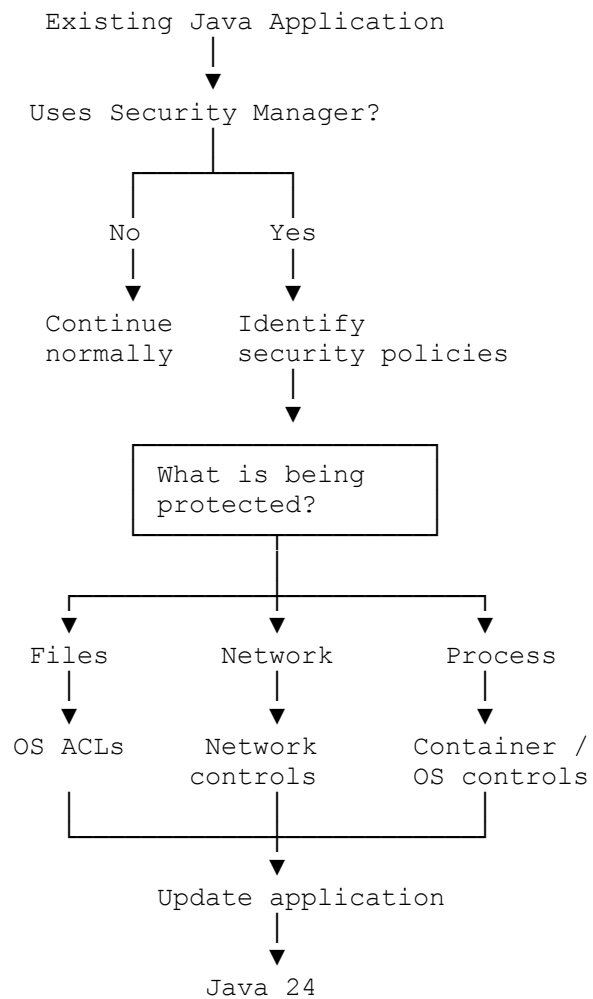
Modern applications more commonly use:



Java 24 therefore completes the long-running process of removing the Security Manager.

Migration Flowchart

For an old application that depends on the Security Manager:

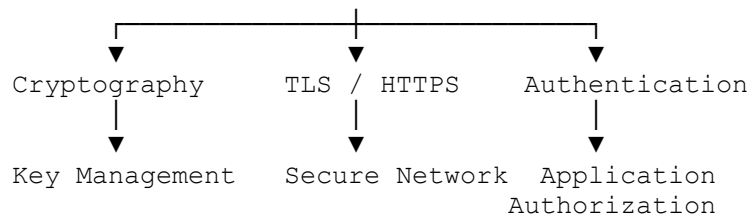


Important Difference

Permanently disabling the Security Manager does NOT mean Java has no security features.

Java still provides extensive security facilities:

Java Security
|



What has been removed is specifically the **Security Manager sandbox mechanism**.

Java 24 Impact

Area	Java 24
Security Manager	Permanently disabled
System.setSecurityManager()	No longer usable
Security Manager sandbox	Removed
Java language syntax	No change
Cryptography APIs	Still available
TLS/HTTPS	Still available
Application authorization	Still available
OS/container security	Still available
Existing Security Manager applications	Require migration

Key Takeaways

- **Java Version:** 24
- **Year:** 2025
- **JEP:** 486
- **Feature:** Permanently Disable the Security Manager
- **Type:** JVM/platform change
- **Main change:** Security Manager can no longer be enabled
- **Reason:** Complete the long-planned removal of the legacy sandboxing mechanism
- **Java syntax:** No new syntax
- **Migration:** Replace Security Manager-based protection with appropriate application, OS, container, and deployment-level security controls.

One-line definition

Java 24 permanently disables the legacy Security Manager, completing its removal after years of deprecation and requiring applications that depended on Security Manager-based sandboxing to adopt modern security mechanisms such as application authorization, OS permissions, containers, and process isolation.

Java 24 — Scoped Values (Fourth Preview)

Java Version: 24

Year: 2025

Feature: Scoped Values (Fourth Preview)

JEP: 487

UniqueClassName: ScopedValues

Definition

Scoped Values provide a mechanism for safely and efficiently sharing **immutable, context-specific values** with methods executed within a particular scope.

In Java 24, Scoped Values were introduced as a **fourth preview**. They are designed particularly for use with **virtual threads and structured concurrency**, where many threads may need access to the same contextual information.

The main API is:

`java.lang.ScopedValue`

A `ScopedValue<T>` allows a value to be:

1. **Bound** to a value.
2. **Used** by code running inside the binding's scope.
3. **Automatically unavailable** when execution leaves that scope.

Basic Idea

Without Scoped Values, context is often passed explicitly:

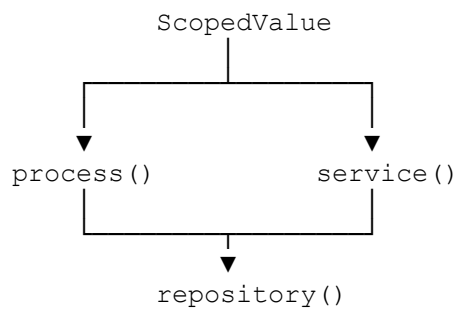
`main()`
↓
▼

process(userId)
↓
service(userId)
↓
repository(userId)

Every method needs to receive the context:

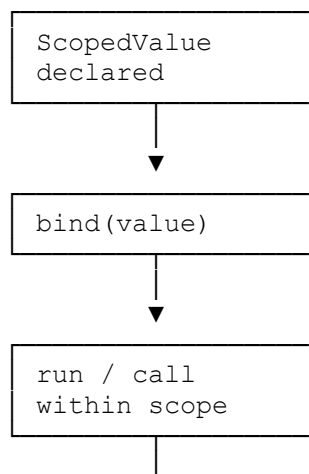
```
process(userId);  
service(userId);  
repository(userId);
```

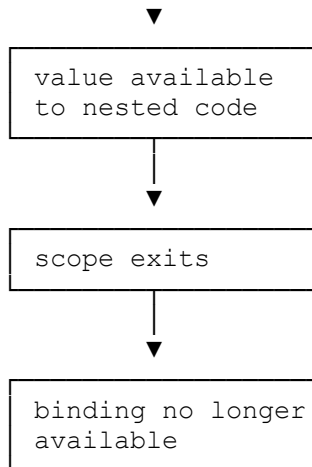
With a Scoped Value:



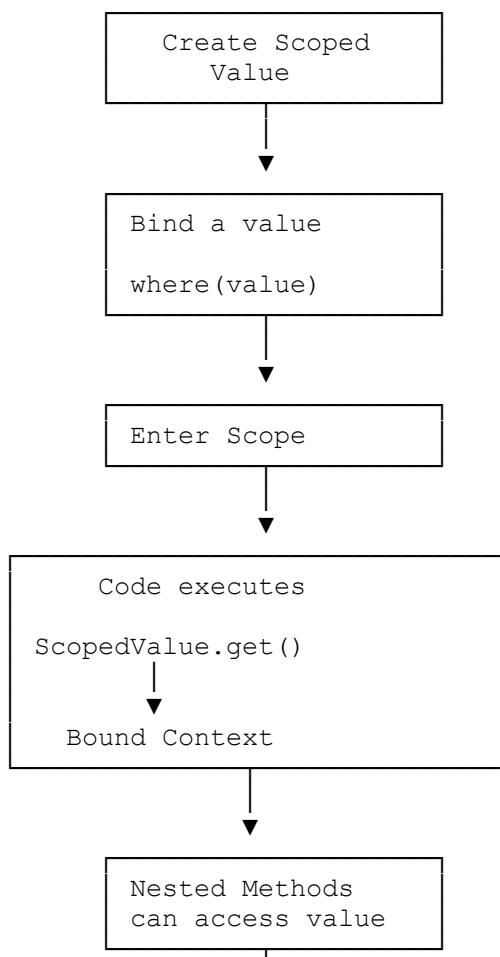
The methods can retrieve the contextual value directly from the scoped value.

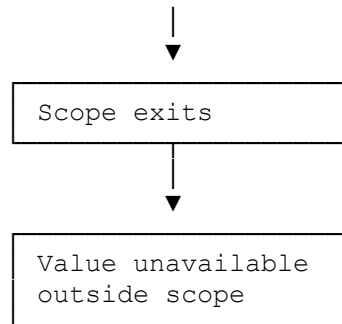
Scoped Value Lifecycle





Flowchart





Java Code Example

Here is a simple Java 24 example.

```
public class ScopedValues {

    private static final ScopedValue<String> USER =
        ScopedValue.newInstance();

    public static void main(String[] args) {

        ScopedValue.where(USER, "Anurag")
            .run(() -> {

                System.out.println(
                    "Main: " + USER.get()
                );

                processRequest();
            });
    }

    static void processRequest() {

        System.out.println(
            "Processing request for: "
            + USER.get()
        );

        accessService();
    }

    static void accessService() {

        System.out.println(
            "Service user: "
            + USER.get()
        );
    }
}
```

Output

```
Main : Anurag
Processing request for : Anurag
Service user : Anurag
```

Notice that USER is not passed as a method parameter:

```
processRequest();
```

and:

```
accessService();
```

Yet both methods can access:

```
USER.get()
```

because they execute inside the scoped binding.

Understanding the Code

1. Create the Scoped Value

```
private static final ScopedValue<String> USER =
    ScopedValue.newInstance();
```

This creates a scoped value capable of holding a String.

```
ScopedValue<String>
```



```
USER
```



```
Context value
```

2. Bind a Value

```
ScopedValue.where(USER, "Anurag")
```

This establishes:

USER
↓
"Anurag"

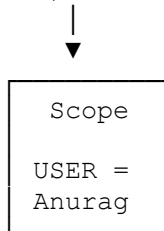
for the scope that follows.

3. Enter the Scope

```
.run() -> {  
  ...  
};
```

Everything inside this lambda executes while the value is bound.

```
where (USER, "Anurag")
```



4. Read the Value

```
USER.get()
```

returns:

"Anurag"

as long as the code is executing within the appropriate scope.

Scope Boundary

An important property is that the binding is **scoped**.

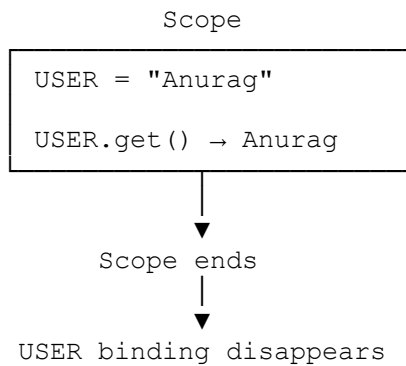
```
ScopedValue.where(USER, "Anurag")  
  .run() -> {
```

```
System.out.println(USER.get());

});

// USER is not bound here
```

Conceptually:



This prevents contextual state from accidentally remaining available indefinitely.

Nested Scopes

Scoped Values can be rebound in a nested scope.

```
public class ScopedValues {

    static final ScopedValue<String> USER =
        ScopedValue.newInstance();

    public static void main(String[] args) {

        ScopedValue.where(USER, "Anurag")
            .run(() -> {

                System.out.println(
                    "Outer: " + USER.get()
                );

                ScopedValue.where(USER, "Rahul")
                    .run(() -> {

                        System.out.println(
                            "Inner: " + USER.get()
                        );
                    });
            });
    }
}
```

```

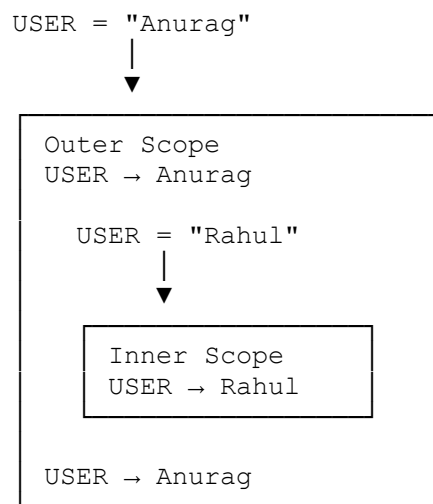
        System.out.println(
            "Outer again: " + USER.get()
        );
    }
}

```

Output:

Outer: Anurag
 Inner: Rahul
 Outer again: Anurag

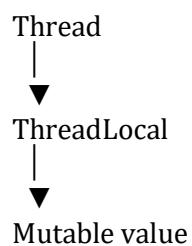
The inner binding temporarily shadows the outer binding:



Scoped Values vs ThreadLocal

Scoped Values are particularly useful as an alternative to some uses of ThreadLocal.

ThreadLocal



↓
▼
Must manage lifecycle carefully

ScopedValue

Execution Scope

↓
▼
ScopedValue

↓
▼
Immutable context

↓
▼
Automatically ends with scope

Feature	ThreadLocal	ScopedValue
Main purpose	Per-thread state	Scoped context
Mutability	Can be mutable	Designed for immutable values
Lifetime	Associated with thread	Associated with dynamic scope
Explicit cleanup	Often necessary	Scope controls lifetime
Nested rebinding	Possible	Natural
Virtual-thread use	Can be costly depending on usage	Designed with modern concurrency in mind
Structured concurrency	Not specifically designed for it	Designed to work well with it

Example: Request Context

A practical use case is carrying a request ID through several layers.

```
public class ScopedValues {  
  
    static final ScopedValue<String> REQUEST_ID =  
        ScopedValue.newInstance();  
  
}
```

```

public static void main(String[] args) {

    ScopedValue.where(
        REQUEST_ID,
        "REQ-2025-001"
    ).run(() -> {

        controller();
    });
}

static void controller() {

    System.out.println(
        "Controller: "
        + REQUEST_ID.get()
    );

    service();
}

static void service() {

    System.out.println(
        "Service: "
        + REQUEST_ID.get()
    );

    repository();
}

static void repository() {

    System.out.println(
        "Repository: "
        + REQUEST_ID.get()
    );
}
}

```

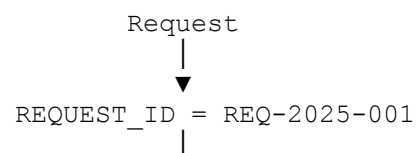
Output:

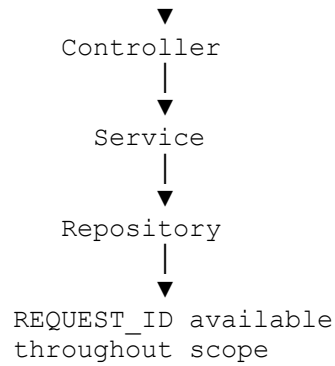
```

Controller      : REQ-2025-001
Service         : REQ-2025-001
Repository      : REQ-2025-001

```

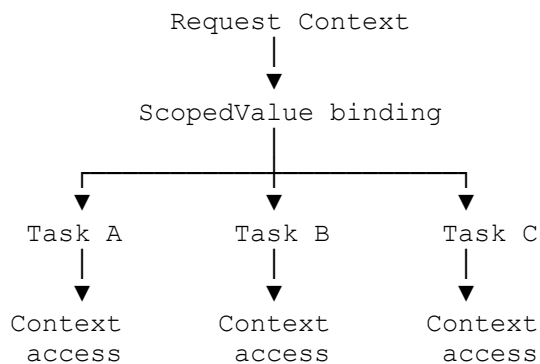
The request ID flows through the call hierarchy without being passed through every method parameter.





Why Scoped Values Are Useful With Concurrency

Scoped Values are particularly relevant to Java's modern concurrency model.



This makes them useful for passing context such as:

- Request IDs
- Authentication/context information
- Transaction identifiers
- Logging context
- Configuration context
- Correlation IDs

The value should generally be treated as **immutable context**, rather than mutable shared state.

Java 24 Preview Syntax

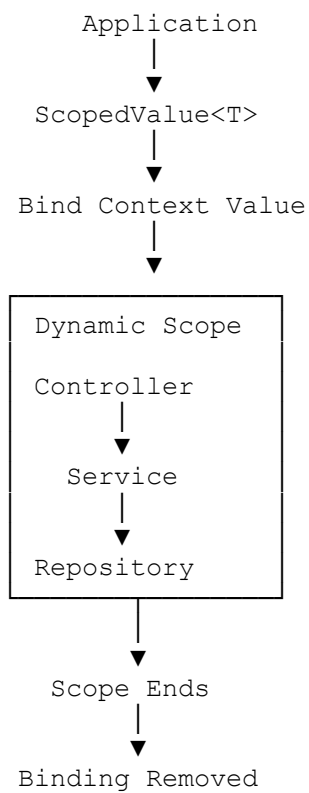
Because this is a **preview feature**, Java 24 requires preview support when compiling/running code using it.

javac --enable-preview --release 24 ScopedValues.java

Run:

java --enable-preview ScopedValues

Overall Architecture



Key Takeaways

Property	Details
Java Version	24
Year	2025
JEP	487
Status	Fourth Preview

Main API	java.lang.ScopedValue
Creation	ScopedValue.newInstance()
Binding	ScopedValue.where(...)
Reading	ScopedValue.get()
Primary purpose	Share immutable contextual data
Scope	Dynamic execution scope
Mutation model	Designed for immutable values
Concurrency	Especially useful with modern Java concurrency
Java syntax changes	No new language syntax

One-line definition

Java 24 Scoped Values (JEP 487) provide a safe, scoped mechanism for sharing immutable contextual data across methods and concurrent tasks without explicitly passing that data through every method parameter, with the binding automatically ending when its scope exits.

Java 24 — Primitive Types in Patterns (Second Preview)

Java Version: 24

Year: 2025

Feature: Primitive Types in Patterns (Second Preview)

JEP: 488

UniqueClassName: PrimitiveTypesPatterns

Definition

Primitive Types in Patterns is a **second-preview Java language feature in Java 24** that extends **pattern matching** so that primitive values such as int, long, double, etc. can participate directly in more pattern constructs, including instanceof and switch.

The feature builds on Java's pattern-matching evolution and makes it possible to express conditions involving primitive types more naturally.

For example, Java 24 allows patterns such as:

```
if (obj instanceof int i) {  
    System.out.println(i);  
}
```

and primitive type patterns in switch:

```
switch (value) {  
    case int i -> System.out.println("Integer: " + i);  
    case long l -> System.out.println("Long: " + l);  
    default -> System.out.println("Other");  
}
```

The feature also addresses **primitive type conversions in patterns**, including safe handling of conversions that might otherwise lose information.

Traditional Approach

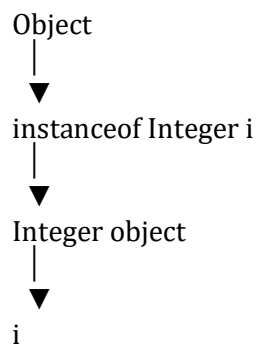
Before primitive type patterns, pattern matching was mainly focused on **reference types**.

For example:

```
Object value = Integer.valueOf(100);
```

```
if (value instanceof Integer i) {  
    System.out.println(i);  
}
```

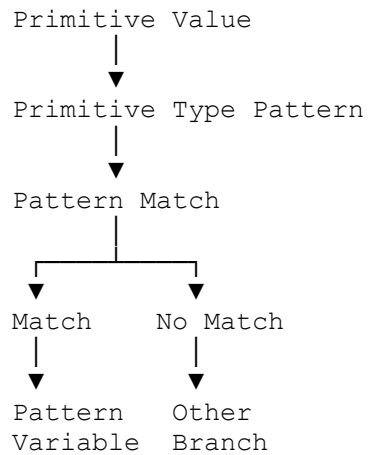
The pattern is based on the wrapper/reference type:



Primitive type patterns extend this model.

Primitive Type Pattern

Conceptually:



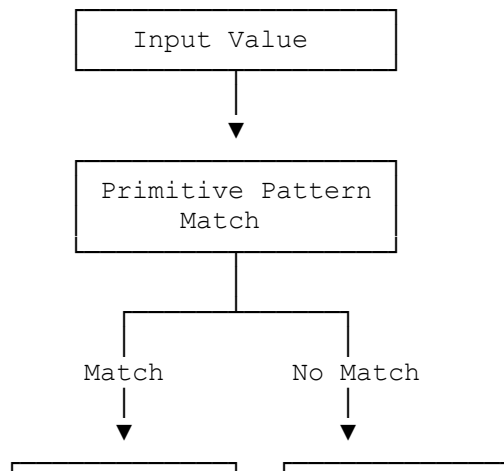
For example:

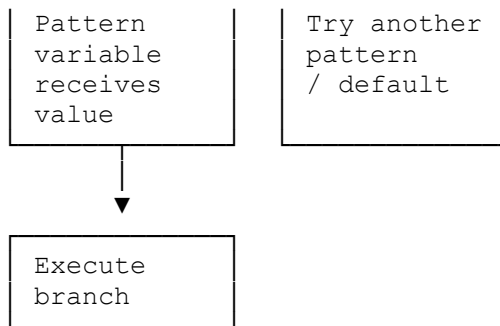
```
int value = 100;
```

```
if (value instanceof int i) {  
    System.out.println(i);  
}
```

Here i represents the matched primitive int value.

Flowchart





Java Code Example 1 — Primitive instanceof

```
public class PrimitiveTypesPatterns {  
    public static void main(String[] args) {  
        int value = 100;  
        if (value instanceof int i) {  
            System.out.println(  
                "Matched int: " + i  
            );  
        }  
    }  
}
```

Output:

Matched int: 100

The important part is:

`value instanceof int i`

The pattern consists of:

```
int  
└─ Primitive type  
  
i  
└─ Pattern variable
```

Java Code Example 2 — Primitive Types in switch

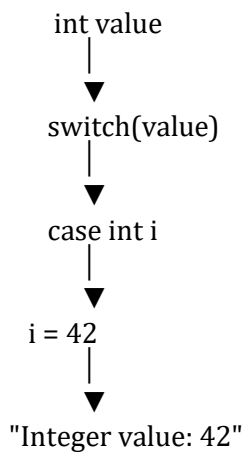
Primitive patterns become especially useful with switch.

```
public class PrimitiveTypesPatterns {  
    public static void main(String[] args) {  
        int value = 42;  
  
        String result = switch (value) {  
            case int i ->  
                "Integer value: " + i;  
  
            default ->  
                "Other value";  
        };  
  
        System.out.println(result);  
    }  
}
```

Output:

Integer value: 42

Conceptually:



Java Code Example 3 — Primitive Type Conversion

One important aspect of JEP 488 is that primitive patterns can participate in conversions.

For example:

```
public class PrimitiveTypesPatterns {  
    public static void main(String[] args) {  
        int value = 100;  
        switch (value) {  
            case byte b ->  
                System.out.println(  
                    "byte: " + b  
                );  
            case int i ->  
                System.out.println(  
                    "int: " + i  
                );  
            default ->  
                System.out.println(  
                    "Other"  
                );  
        }  
    }  
}
```

The JVM/compiler must consider whether a value can be represented by the primitive type used by the pattern.

The feature therefore makes primitive pattern matching more expressive while preserving Java's rules around numeric conversions.

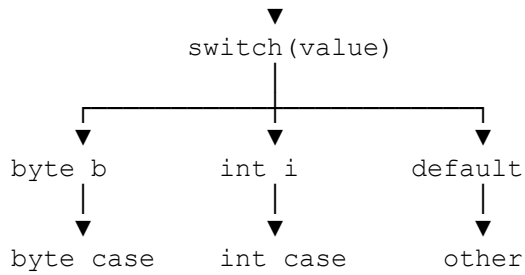
Primitive Type Patterns With switch

Consider:

```
int value = 10;
```

A primitive pattern can be used as part of a switch:

```
value  
|
```

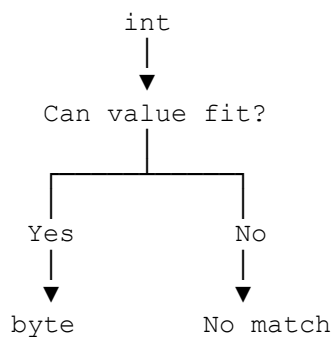


This allows developers to express numeric type-based logic directly in the pattern language.

Narrowing Conversion

The feature is particularly interesting when patterns involve a primitive type that has a smaller range.

For example:



For an int value:

100

it can fit into a byte.

But:

1000

cannot fit into a Java byte.

The pattern mechanism takes the representable range into account rather than blindly performing an unchecked narrowing conversion.

Primitive Patterns and null

Primitive values themselves cannot be null.

```
int x = 10;
```

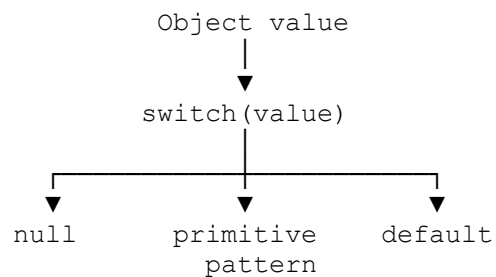
always contains a value.

But when dealing with a reference such as:

```
Object value = null;
```

the switch pattern model can distinguish null from primitive/reference patterns.

Conceptually:



Why Is This Useful?

Primitive types are extremely common in Java:

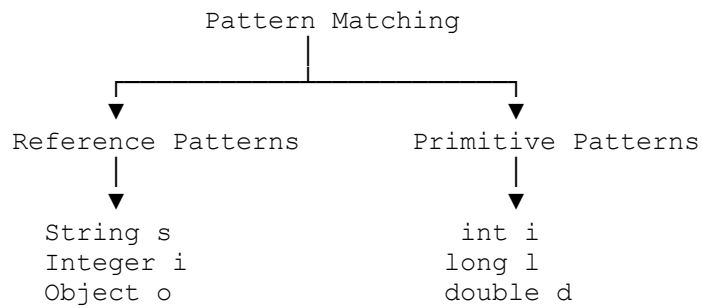
```
byte  
short  
int  
long  
float  
double  
char  
boolean
```

Historically, pattern matching was primarily designed around reference types:

```
String  
Integer  
Long  
Double  
Object
```

...

Primitive Types in Patterns brings primitives into the pattern-matching model.



Comparison

Traditional

```
if (value instanceof Integer i) {  
    ...  
}
```

Requires a reference/wrapper type.

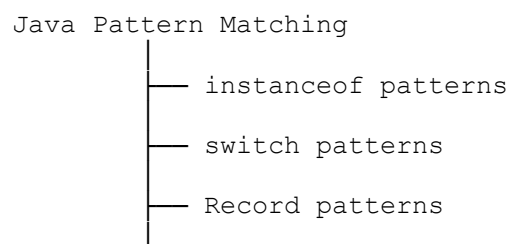
Primitive Pattern

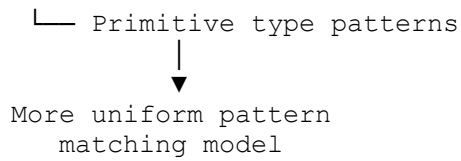
```
if (value instanceof int i) {  
    ...  
}
```

Works directly with the primitive type.

Relationship With Pattern Matching

The evolution can be viewed as:





Java 24 Preview Compilation

Because **Primitive Types in Patterns** is a preview feature in Java 24, compile with preview support:

```
javac --enable-preview --release 24 PrimitiveTypesPatterns.java
```

Run:

```
java --enable-preview PrimitiveTypesPatterns
```

Important Note

This feature should **not** be confused with simply using primitive variables:

```
int x = 10;
```

Primitive variables have existed since Java 1.0.

The new feature concerns **using primitive types as patterns**, for example:

```
case int i -> ...
```

and related pattern constructs.

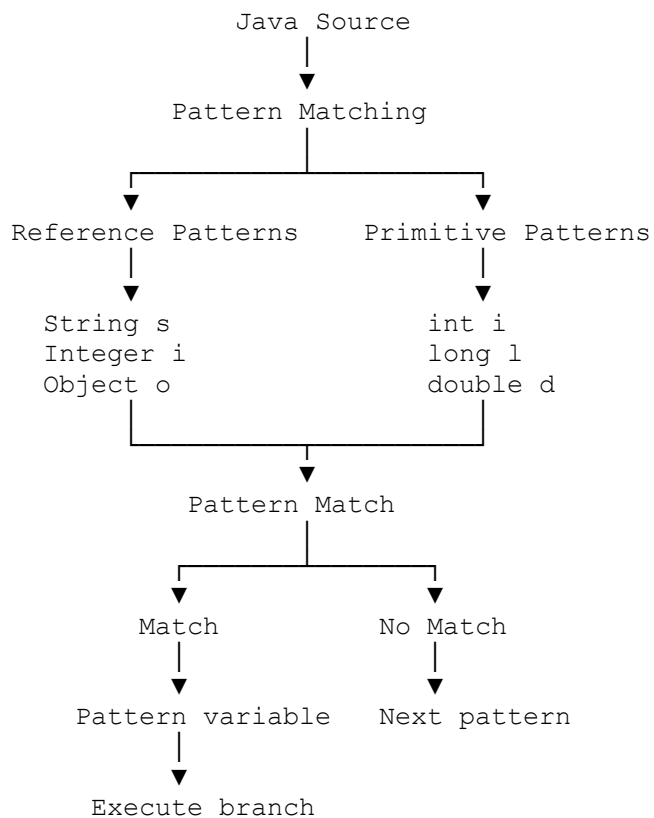
So:

Primitive variable

≠

Primitive type pattern

Overall Architecture



Key Takeaways

Property	Details
Java Version	24
Year	2025
JEP	488
Status	Second Preview
Feature	Primitive Types in Patterns
Main area	Pattern Matching
Primitive types	Can participate directly in patterns
switch	Supports primitive type patterns
instanceof	Extended for primitive patterns

Conversions	Handles primitive conversion rules
Java syntax	New pattern forms
Compilation	Requires --enable-preview in Java 24

One-line definition

Java 24's Primitive Types in Patterns (JEP 488) extends pattern matching to primitive types, allowing constructs such as `case int i` and related primitive patterns while providing well-defined rules for primitive conversions and matching.